



DBMS COURSE PROJECT

Design and Implementation of a Database Management System for Smart Parking Lot Allocation and Billing System

Project 20 · Group 12



Submitted by

Lakshay Sethia	25WU0101167
Sidharth Sood	25WU0101132
Shardul Yadav	25WU0101127
Naman Singla	25WU0101084

Course: Database Management Systems

Faculty Name: Dr. Kiran Mayee Adavala

School of Technology, Woxsen University

Academic Year: 2025-2029

ABSTRACT

Multi-level parking facilities still depend on manual ticketing and rate cards, which leads to two recurring failures: two attendants allocating the same bay because neither can see the other's action, and revenue leakage because charges are worked out by hand and unpaid exits go unrecorded. This project designs and implements **SmartPark**, a relational database system and web application that handles slot allocation, reservations, gate entry and exit, tariff-based billing, parking passes, payments, violations and reporting for multiple facilities.

The database is implemented in **PostgreSQL 17** with 17 tables normalized to Third Normal Form (and BCNF). The business rules are enforced by the database itself rather than by application code: partial unique indexes guarantee one vehicle per bay, composite foreign keys make it impossible to park or book a car in a two-wheeler bay, GiST exclusion constraints prevent overlapping reservations, passes and tariffs, and every charge is computed by a PL/pgSQL function from the tariff in force, with a trigger that overwrites any amount a client tries to supply. Gate operations run as single transactions that lock the chosen bay with `SELECT ... FOR UPDATE SKIP LOCKED`, so concurrent operators can never be given the same bay. Row-level security limits gate operators to their own facility and customers to their own records, while administrators see everything; an audit trigger records every change.

A Python (FastAPI) application connects to the database through `psycopg3` and serves a browser interface for all operations. The system was tested with 25 deliberate attempts to violate the business rules (24 refused by the database and 1 silently corrected, as designed), 11 lifecycle checks, 12 row-level security tests, a 40-check end-to-end test, 18 automated API tests and browser checks on every screen. All of these pass.

Keywords: relational database, PostgreSQL, normalization, constraints, transactions, row-level security, parking management, FastAPI.

TABLE OF CONTENTS

Abstract.....	2
1. Introduction and Problem Statement.....	5
1.1 Introduction.....	5
1.2 Problem Statement.....	5
1.3 Intended Users.....	5
2. Objectives and Scope.....	6
2.1 Objectives.....	6
2.2 Scope.....	6
3. Software and Hardware Requirements.....	7
3.1 Software Requirements.....	7
3.2 Hardware Requirements.....	7
4. ER Diagram.....	8
4.1 Entities and Key Attributes.....	10
4.2 Relationships, Cardinality and Participation.....	11
4.3 How the Slot / Vehicle-Type Rule Is Built into the Model.....	12
4.4 Detailed ER Diagrams with Attributes.....	13
5. Relational Schema and Normalization.....	18
5.1 Relational Schema.....	18
5.2 Functional Dependencies.....	22
5.3 Normalization to Third Normal Form.....	24
5.4 Result.....	25
6. Data Dictionary.....	26
7. SQL Commands Used (DDL and DML) with Sample Outputs.....	35
7.1 Data Definition Language (DDL).....	35
7.2 Data Manipulation Language (DML).....	38
7.3 A Transaction: Gate Entry and Exit.....	39
7.4 Sample Data: Rows in Every Table.....	39
8. Queries with Outputs.....	41
9. UI Design and Screenshots.....	46
9.1 Design Approach.....	46
9.2 Screens.....	46
9.3 Insert, View and Delete — Reflected in the Database.....	58
10. Implementation Details.....	62
10.1 Technology Stack and Architecture.....	62
10.2 Database Connectivity.....	62
10.3 Key Code Snippets.....	63

10.4 Security.....	63
11. Testing.....	65
11.1 Approach.....	65
11.2 Constraint Test Cases.....	65
11.3 The Seven Required Integrity Checks.....	66
11.4 Lifecycle Test Results.....	67
12. Conclusion and Future Enhancements.....	68
12.1 Conclusion.....	68
12.2 Future Enhancements.....	68
13. References.....	69
14. Contribution of Each Member.....	70
Appendix A: GitHub Repository Link.....	71
A.1 Running the Project.....	71

1. Introduction and Problem Statement

1.1 Introduction

Urban parking structures serve hundreds of vehicles a day across several floors, zones and vehicle categories. Each arrival needs a suitable bay, each departure needs a correctly priced bill, and regular users expect reservations and passes. When this is managed with paper tickets and spreadsheets, information about which bay is free, which vehicle is inside and what is owed is scattered and out of date.

A database management system solves this by keeping one authoritative, consistent record of the facility, its bays, its customers and every parking event. This project builds that system: the data model, the rules that keep the data correct, the transactions that change it safely, and an application through which staff and customers use it.

1.2 Problem Statement

Design and implement a database management system for smart parking lot allocation and billing. The system must manage multi-level facilities, floors, zones and slots of different vehicle types; register customers and their vehicles; allow reservations; record gate entries and exits; allocate a free, suitable slot on arrival; compute duration-based charges from versioned tariffs; sell and honour passes; record payments; detect violations; and produce occupancy, revenue, peak-hour, duration, pass-usage, violation and free-slot reports.

Specifically, the system must guarantee that:

- a slot never holds two active vehicles, and a vehicle never occupies two slots at once;
- a vehicle is only ever parked in, or booked into, a slot built for its vehicle type;
- an exit time can never precede its entry time;
- live reservations for a slot never overlap in time, and expired holds are released;
- every charge is derived from the applicable tariff, never typed in by a client, and is never negative.

1.3 Intended Users

Table 1: *Users of the system*

User	Role in the system
Administrator	Manages tariffs, customers and vehicles; sees every facility; reads the audit trail
Gate operator	Records arrivals and departures, records payments and takes bays out of service, at their own facility only
Customer	Views their own vehicles, bookings, passes and bills; makes reservations

2. Objectives and Scope

2.1 Objectives

- Model the parking domain (facilities, floors, zones, slots, vehicle types, customers, vehicles, reservations, sessions, passes, tariffs, bills, payments, violations) as a relational schema normalized to 3NF.
- Enforce every business rule with primary keys, foreign keys, CHECK, UNIQUE and EXCLUDE constraints, triggers and stored functions, so that invalid data cannot be stored whatever the client does.
- Implement allocation, entry, exit and billing as atomic transactions that are safe under concurrency.
- Provide a working user interface connected to the database for insertion, update, deletion and viewing of records, with search, validation and clear error messages.
- Produce the required reports directly from SQL views over live data.
- Restrict each user to the data they are entitled to with role-based row-level security.

2.2 Scope

Table 2: *Project scope*

In scope	Out of scope
Multiple facilities, each with floors, zones and typed slots	Number-plate recognition cameras and barrier hardware
Customer and vehicle registration (create, edit, delete)	Online card or UPI payment processing (payments are recorded, not processed)
Reservations with automatic expiry and no-show detection	Mobile applications
Gate entry with automatic slot allocation; gate exit with automatic billing	Dynamic or surge pricing
Versioned tariffs with free minutes, hourly rates and daily caps	
Daily, weekly and monthly passes	
Payments (cash, card, UPI, wallet, net banking) and bill status	
Violations of five kinds; overstay and reservation no-show are detected automatically	
Seven reports, a live dashboard and an audit trail	

3. Software and Hardware Requirements

3.1 Software Requirements

Table 3: *Software requirements*

Component	Used in this project	Purpose
Database server	PostgreSQL 17	Stores the data and enforces every rule (the course permits MySQL, PostgreSQL or Oracle)
Programming language	Python 3.11	Application layer
Web framework	FastAPI 0.141, Uvicorn 0.52	HTTP API serving the interface
Database driver	psycopg 3.3 with psycopg-pool	Connectivity and connection pooling
Security libraries	bcrypt 5.0, PyJWT 2.14	Password hashing and sign-in tokens
Front end	HTML5, CSS3, JavaScript (ES modules); Motion 11 (animation)	User interface; no build step
Operating system	macOS, Windows or Linux	Any system that runs PostgreSQL and Python
Browser	Chrome, Edge, Firefox or Safari (current)	Using the application
Tools	psql, Git and GitHub, a code editor	Running SQL, version control, editing
Testing	pytest 9, Playwright 1.62	Automated API and browser tests
Hosting (demo)	Vercel and Supabase (PostgreSQL 17)	Public online copy of the system

3.2 Hardware Requirements

Table 4: *Hardware requirements*

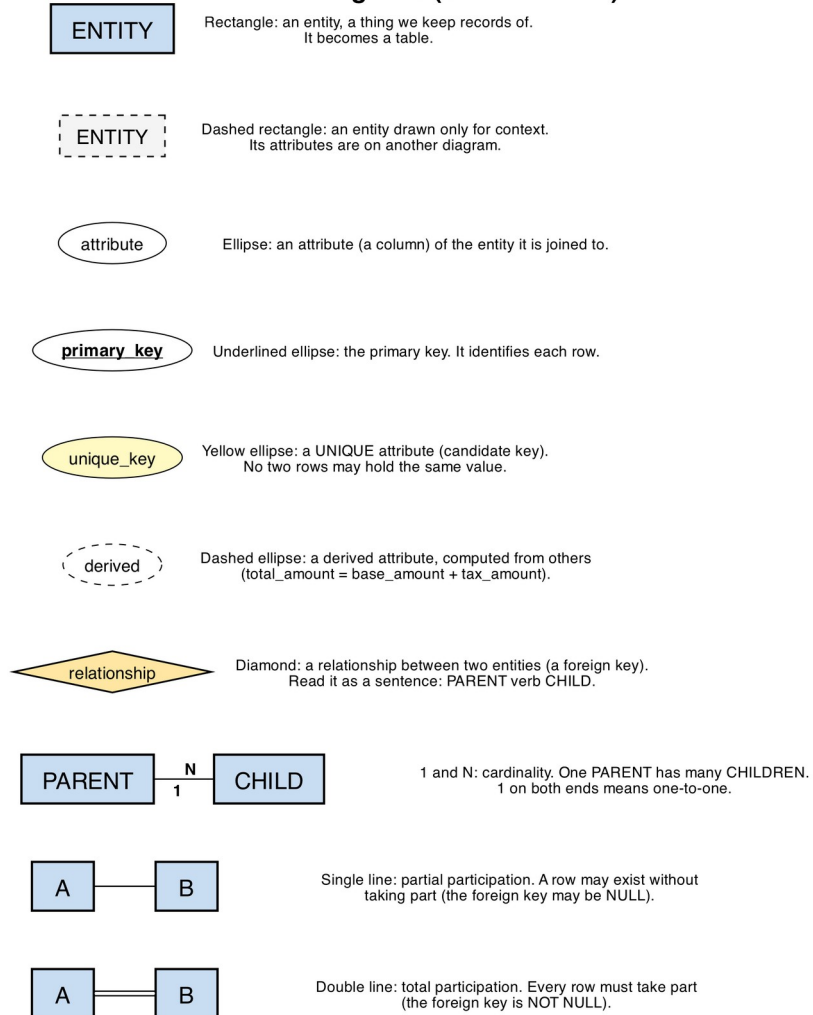
Component	Minimum	Recommended
Processor	Dual-core 64-bit CPU	Quad-core (e.g. Intel Core i5 / Apple M-series)
Memory (RAM)	4 GB	8 GB or more
Storage	1 GB free	2 GB free (database, Python environment, browser)
Display	1280 × 720	1440 × 900 or larger (the interface is designed for laptops)
Network	Not required locally	Internet only for the hosted demo

4. ER Diagram

The ER model contains 17 entities. Fourteen are the entities the problem names (facility, floor, zone, slot, vehicle type, customer, vehicle, reservation, parking session, parking pass, tariff, bill, payment, violation); three more are required by the design: **pass_type** (the pass products on sale), **app_user** (sign-in accounts and roles) and **audit_log** (the change history).

The diagrams use Chen notation. Rectangles are entities, ellipses are attributes (underlined is the primary key, yellow is UNIQUE, dashed is derived) and diamonds are relationships. A 1 or N on a line is the cardinality, and a double line means total participation: every row of that entity must take part, because its foreign key is NOT NULL. A single line means participation is optional. Foreign-key columns are not drawn as attributes, because the relationship diamond is the foreign key. Figure 1 is the key to every shape, Figure 2 shows all 17 entities and 31 relationships, and Section 4.4 gives the same model with every attribute.

How to read the ER diagrams (Chen notation)



Foreign-key columns are not drawn as attributes: the relationship diamond is the foreign key.
Not used here: weak entities (double rectangle) and multivalued attributes (double ellipse). Every entity has its own primary key, and repeating groups such as payments were moved into their own entity during normalization.

Figure 1: Key to the ER diagrams: what every shape and line means

ER diagram: all 17 entities and 31 relationships (their attributes are on the module diagrams)

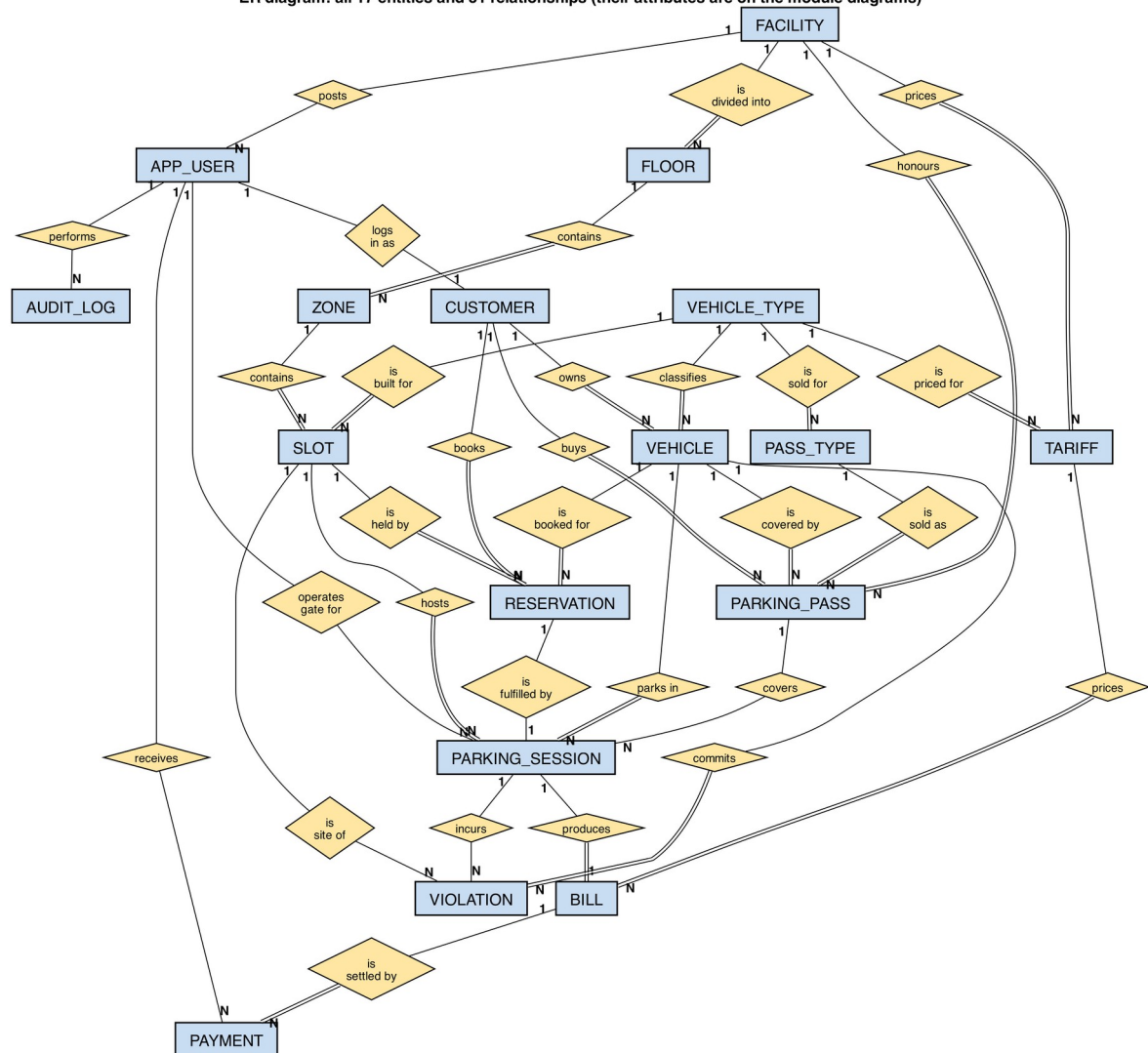


Figure 2: ER diagram overview: all 17 entities and 31 relationships in Chen notation

4.1 Entities and Key Attributes

Table 5: Entities, keys and attributes

Entity	Primary key	Other attributes
FACILITY	facility_id	name (UK), address_line, city, opens_at, closes_at, tax_rate_pct, is_active, created_at
FLOOR	floor_id	facility_id (FK), level_number, name
ZONE	zone_id	floor_id (FK), code, name
SLOT	slot_id	zone_id (FK), code, vehicle_type_id (FK), is_active, grid_row, grid_col, created_at, service_note
VEHICLE_TYPE	vehicle_type_id	code (UK), name, footprint_units
APP_USER	user_id	email (UK), password_hash, full_name, role, facility_id (FK), is_active, created_at

Entity	Primary key	Other attributes
CUSTOMER	customer_id	user_id (FK, UK), full_name, phone (UK), email (UK), created_at
VEHICLE	vehicle_id	customer_id (FK), plate_number (UK), vehicle_type_id (FK), make, model, colour, created_at
TARIFF	tariff_id	facility_id (FK), vehicle_type_id (FK), name, free_minutes, first_hour_rate, subsequent_hour_rate, daily_cap, effective_from, effective_to
PASS_TYPE	pass_type_id	code (UK), name, duration_days, price, vehicle_type_id (FK)
PARKING_PASS	pass_id	customer_id (FK), vehicle_id (FK), pass_type_id (FK), facility_id (FK), valid_from, valid_to, price_paid, cancelled_at, created_at
RESERVATION	reservation_id	customer_id (FK), vehicle_id (FK), slot_id (FK), reserved_from, reserved_until, status, created_at, vehicle_type_id (FK)
PARKING_SESSION	session_id	ticket_no (UK), slot_id (FK), vehicle_id (FK), vehicle_type_id (FK), entry_time, exit_time, reservation_id (FK, UK), pass_id (FK), entry_operator_id (FK), exit_operator_id (FK)
BILL	bill_id	session_id (FK, UK), tariff_id (FK), billable_minutes, base_amount, tax_amount, total_amount, status, generated_at
PAYMENT	payment_id	bill_id (FK), amount, method, reference_no, paid_at, received_by (FK)
VIOLATION	violation_id	kind, session_id (FK), vehicle_id (FK), slot_id (FK), detected_at, penalty_amount, notes, resolved_at
AUDIT_LOG	audit_id	occurred_at, actor_user_id (FK), actor_role, table_name, row_id, action, changes

4.2 Relationships, Cardinality and Participation

Table 6: Relationships with cardinality and participation

Relationship	Cardinality	Participation / enforced by
Facility – Floor	1 : N	Every floor belongs to exactly one facility (total); a facility may have no floors yet (partial)
Floor – Zone, Zone – Slot	1 : N	Containment chain; every slot resolves to exactly one facility
Vehicle type – Slot / Vehicle / Tariff / Pass type	1 : N	Every slot, vehicle, tariff and pass type has exactly one vehicle type (total)
Facility – Tariff / Parking pass	1 : N	Every tariff and pass belongs to one facility (total)
Facility – App user	0..1 : N	Only operators are posted to a facility; facility_id is NULL for admins and customers
App user – Customer	0..1 : 0..1	A customer may have one login (walk-in customers have none); user_id is UNIQUE
Customer – Vehicle / Reservation / Parking pass	1 : N	Each has exactly one customer (total); a customer may have none
Vehicle – Parking session	1 : N over time, 1 : 1 at any instant	Partial unique index on open sessions
Slot – Parking session	1 : N over time, 1 : 1 at any instant	Partial unique index on open sessions
Slot – Reservation	1 : N, non-overlapping while live	GiST exclusion constraint
Reservation – Parking session	0..1 : 0..1	An arrival fulfils at most one reservation; most

Relationship	Cardinality	Participation / enforced by
		sessions have none (reservation_id is NULL and UNIQUE)
Parking pass – Parking session	0..1 : N	A session is either covered by one pass or charged by tariff (pass_id is NULL)
Parking session – Bill	1 : 0..1	One bill per session, raised at exit; an open session has none yet
Tariff – Bill	1 : N	Every bill records the tariff that priced it
Bill – Payment	1 : N	A bill may be settled in several payments
Vehicle – Violation	1 : N	Every violation names a vehicle (total)
Parking session / Slot – Violation	0..1 : N	Optional: a reservation no-show has no session
App user – Payment / Parking session / Audit log	0..1 : N	The staff member who took the payment, worked the gate or made the change; NULL if not recorded

4.3 How the Slot / Vehicle-Type Rule Is Built into the Model

The rule that a car cannot use a two-wheeler bay is not checked by code. slot and vehicle each declare `UNIQUE (id, vehicle_type_id)`, and parking_session (and reservation) carry vehicle_type_id with two composite foreign keys, one to each. A session row can only exist if both referenced pairs exist, which forces the slot's type and the vehicle's type to be equal.

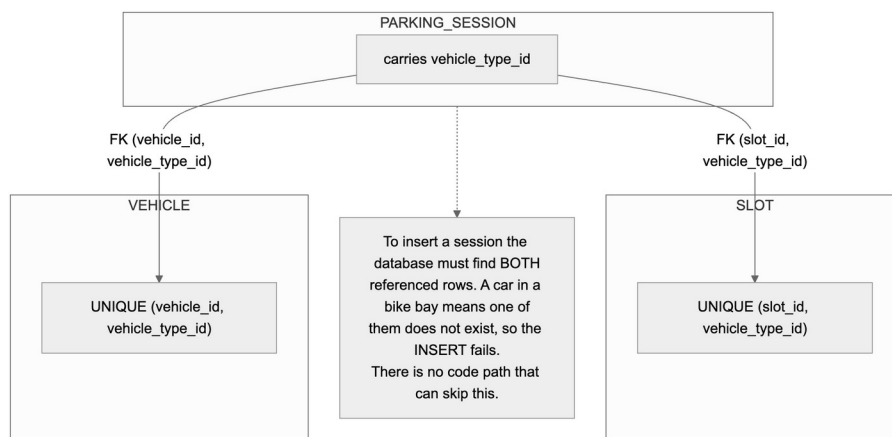


Figure 3: Composite foreign keys enforcing the slot / vehicle-type match

4.4 Detailed ER Diagrams with Attributes

Each diagram shows the entities of one part of the model with all their attributes. A dashed box is an entity drawn only for context; its own attributes are on another diagram. The diagrams are generated from the database itself (tests/gen_schema_docs.py), so cardinality and participation come from the real constraints. Zoom into the PDF for detail.

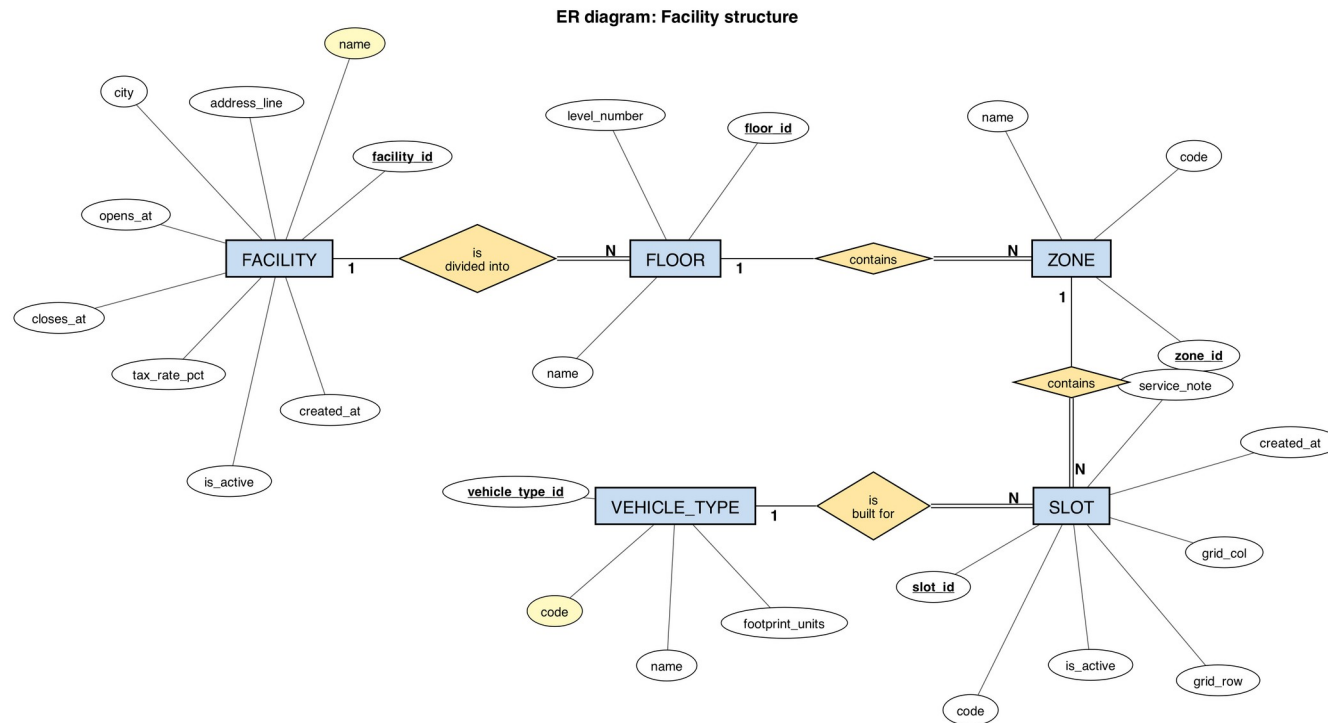


Figure 4: Facility structure: facility, floor, zone, slot and vehicle type

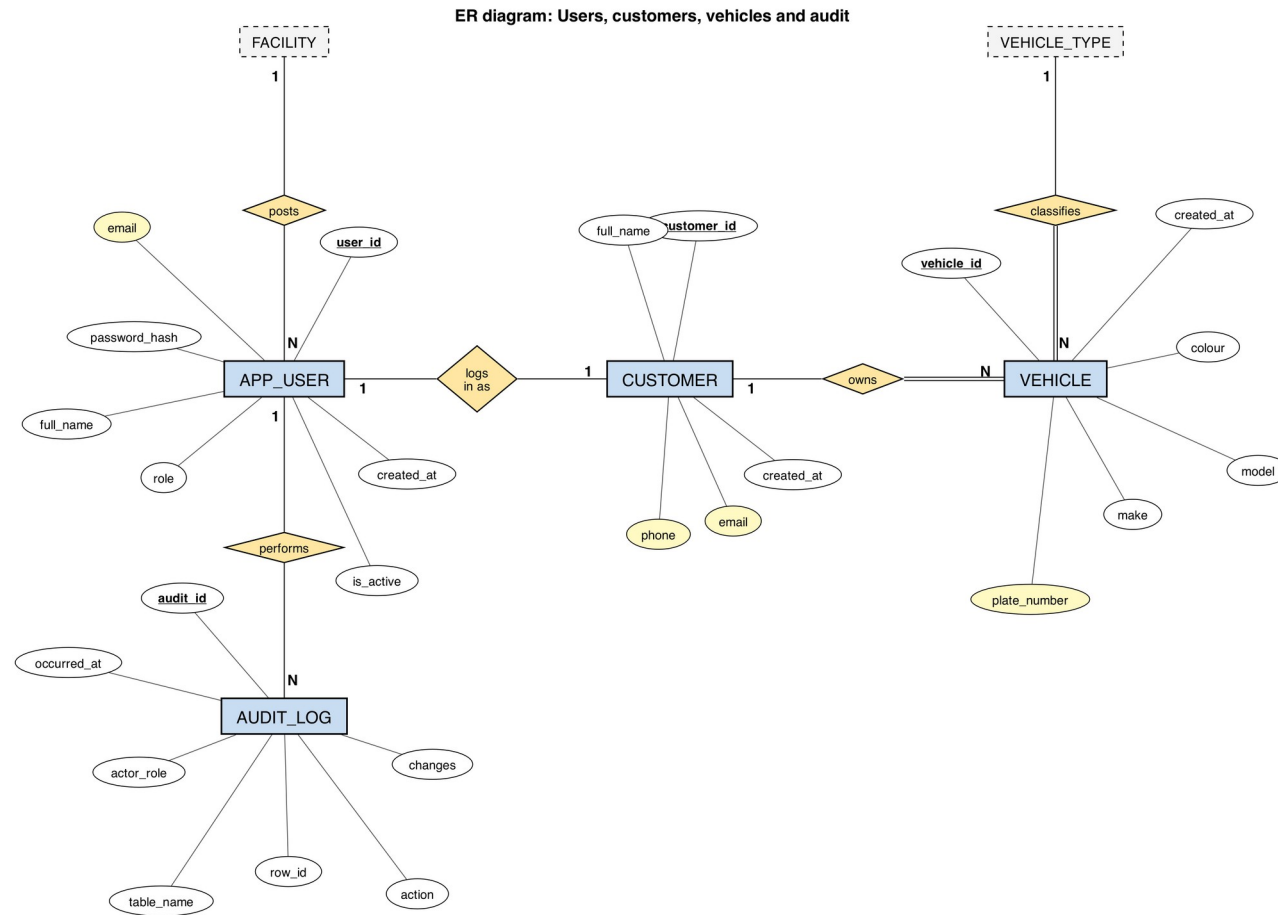


Figure 5: Users, customers, vehicles and the audit trail

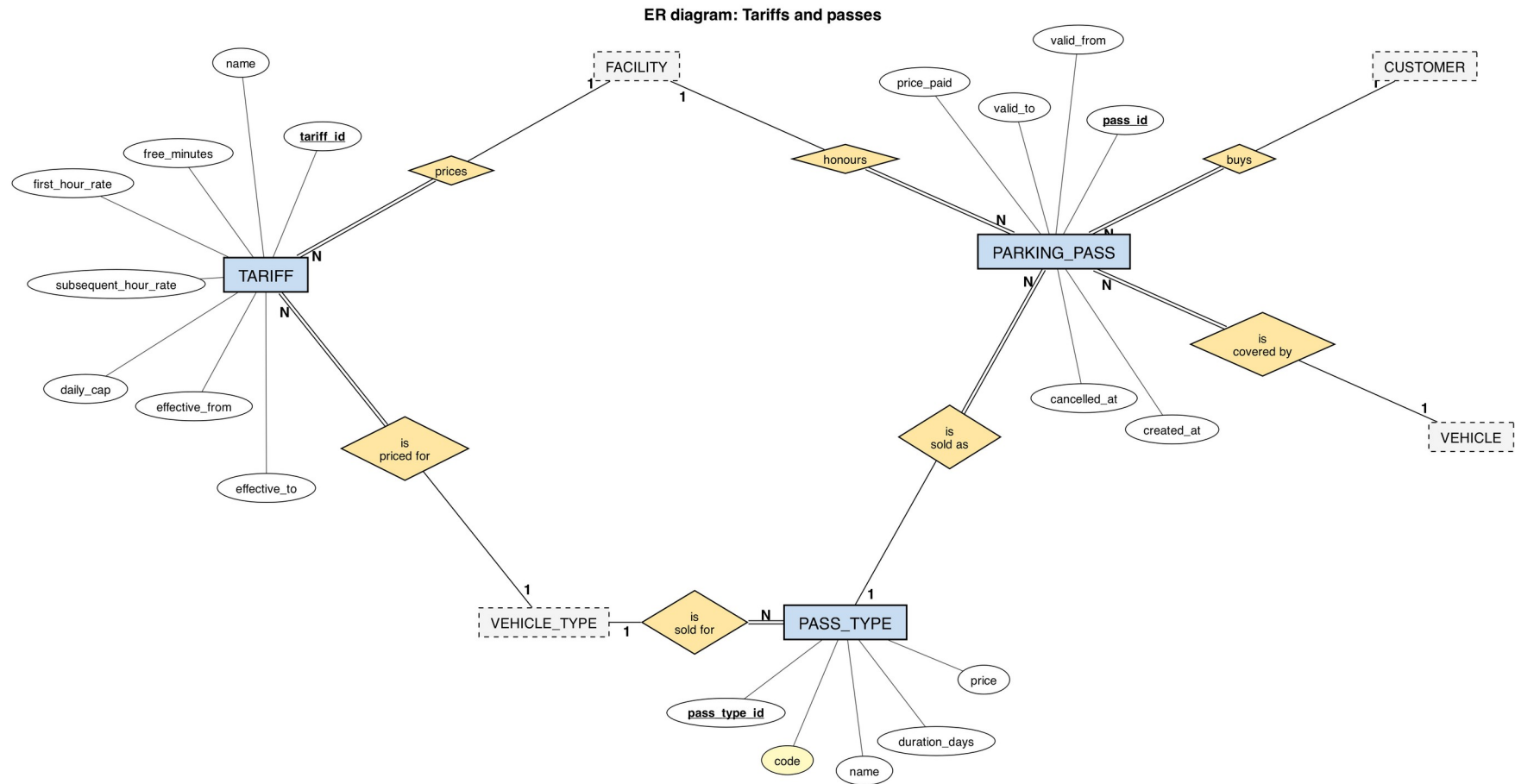


Figure 6: Tariffs and passes

ER diagram: Reservations and parking sessions

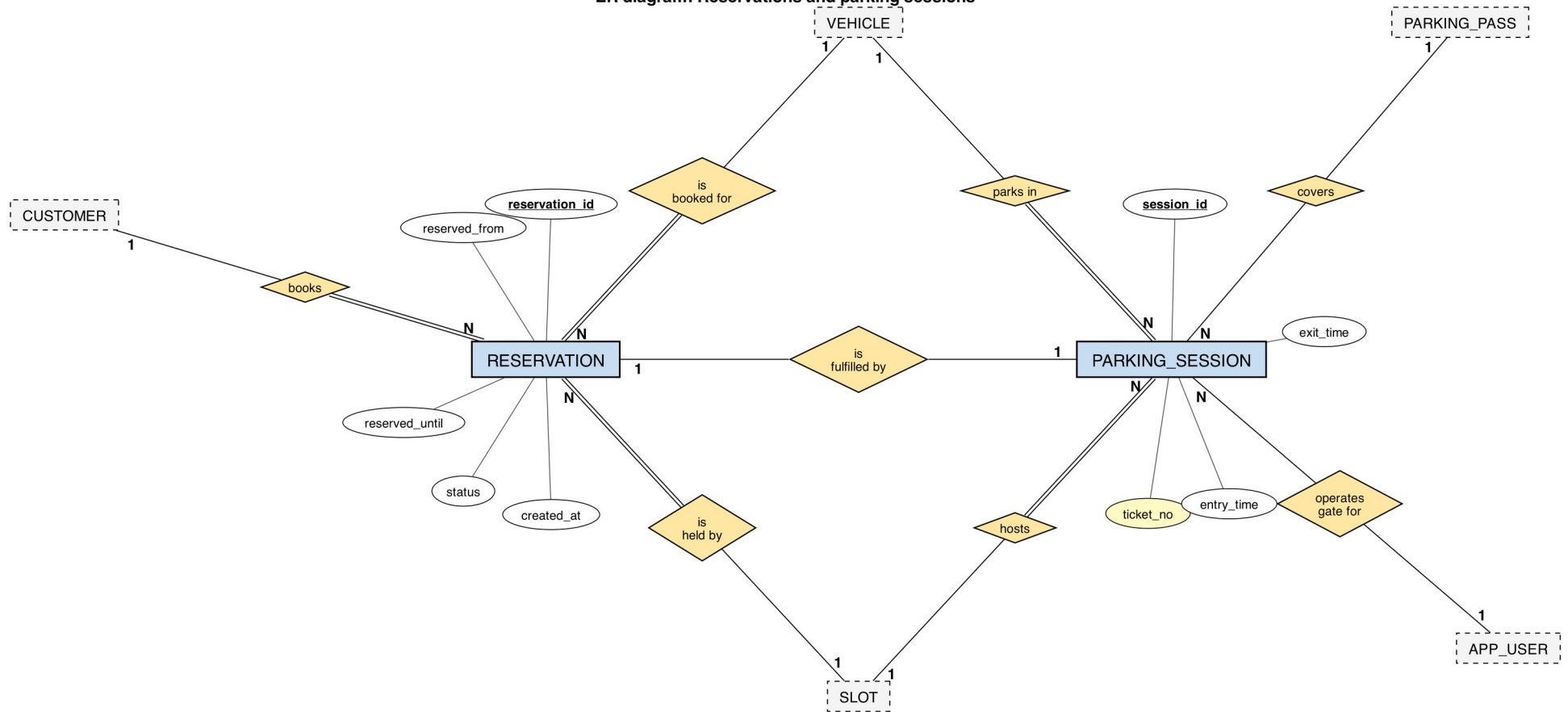


Figure 7: Reservations and parking sessions

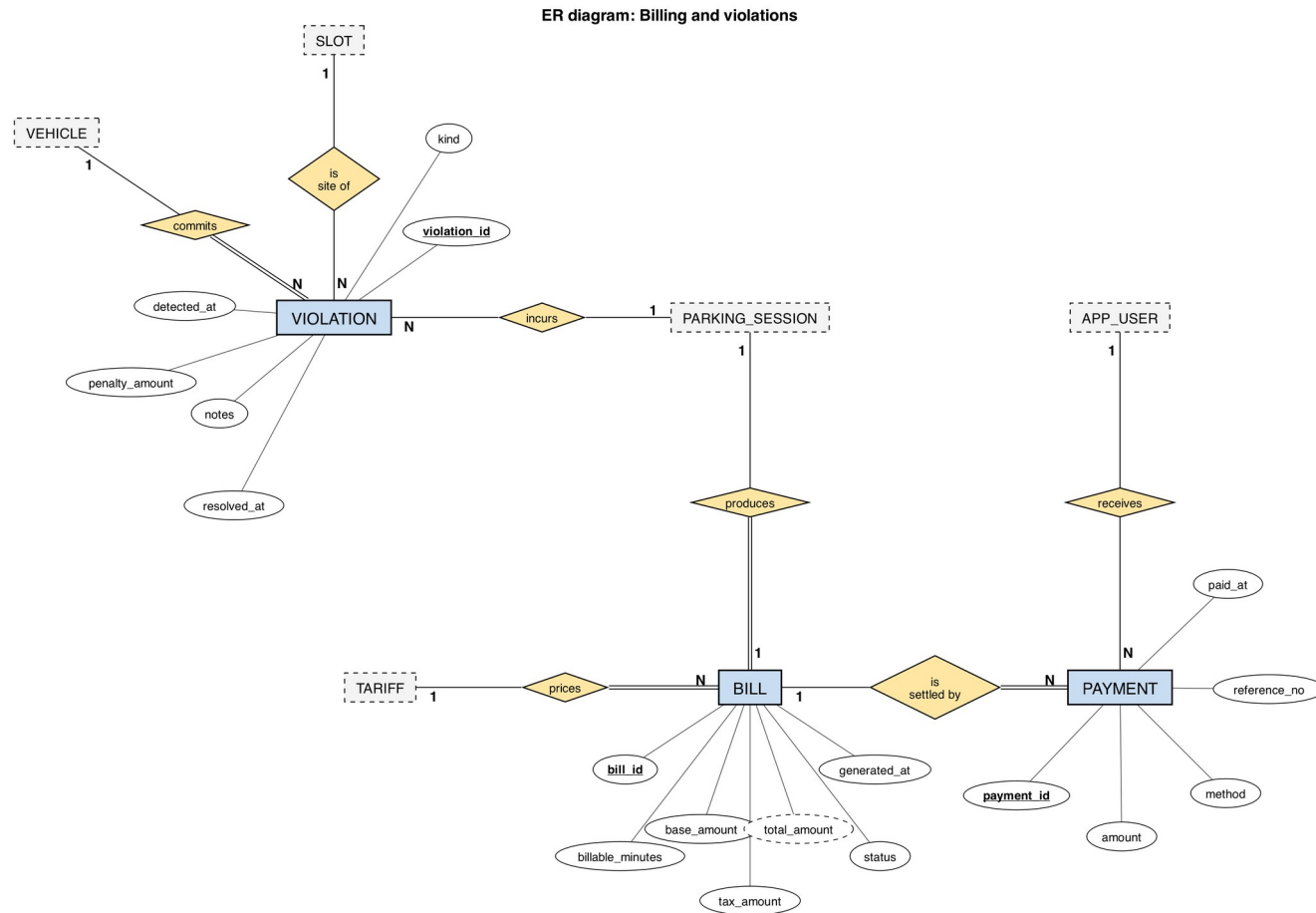


Figure 8: Billing and violations: bills, payments and violations

5. Relational Schema and Normalization

5.1 Relational Schema

Primary keys are **underlined**; foreign keys are shown in *italics* with the table they reference. Every primary key is a surrogate BIGINT GENERATED ALWAYS AS IDENTITY; natural candidate keys (plate number, ticket number, e-mail) are declared UNIQUE.

FACILITY (facility_id, name, address_line, city, opens_at, closes_at, tax_rate_pct, is_active, created_at)

FLOOR (floor_id, *facility_id* → facility, level_number, name)

ZONE (zone_id, *floor_id* → floor, code, name)

VEHICLE_TYPE (vehicle_type_id, code, name, footprint_units)

SLOT (slot_id, *zone_id* → zone, code, *vehicle_type_id* → vehicle_type, is_active, grid_row, grid_col, created_at, service_note)

APP_USER (user_id, email, password_hash, full_name, role, *facility_id* → facility, is_active, created_at)

CUSTOMER (customer_id, *user_id* → app_user, full_name, phone, email, created_at)

VEHICLE (vehicle_id, *customer_id* → customer, plate_number, *vehicle_type_id* → vehicle_type, make, model, colour, created_at)

TARIFF (tariff_id, *facility_id* → facility, *vehicle_type_id* → vehicle_type, name, free_minutes, first_hour_rate, subsequent_hour_rate, daily_cap, effective_from, effective_to)

PASS_TYPE (pass_type_id, code, name, duration_days, price, *vehicle_type_id* → vehicle_type)

PARKING_PASS (pass_id, *customer_id* → customer, *vehicle_id* → vehicle, *pass_type_id* → pass_type, *facility_id* → facility, valid_from, valid_to, price_paid, cancelled_at, created_at)

RESERVATION (reservation_id, *customer_id* → customer, *vehicle_id* → vehicle, *slot_id* → slot, reserved_from, reserved_until, status, created_at, *vehicle_type_id* → slot & vehicle (composite))

PARKING_SESSION (session_id, ticket_no, *slot_id* → slot (composite), *vehicle_id* → vehicle (composite), *vehicle_type_id* → slot & vehicle (composite), entry_time, exit_time, *reservation_id* → reservation, *pass_id* → parking_pass, *entry_operator_id* → app_user, *exit_operator_id* → app_user)

BILL (bill_id, *session_id* → parking_session, *tariff_id* → tariff, billable_minutes, base_amount, tax_amount, total_amount, status, generated_at)

PAYMENT (payment_id, *bill_id* → bill, amount, method, reference_no, paid_at, received_by → app_user)

VIOLATION (violation_id, kind, *session_id* → parking_session, *vehicle_id* → vehicle, *slot_id* → slot, detected_at, penalty_amount, notes, resolved_at)

AUDIT_LOG (audit_id, occurred_at, *actor_user_id* → app_user, actor_role, table_name, row_id, action, changes)

The diagram below draws the same 17 relations as boxes. Each arrow runs from a foreign key

to the primary key it references, and the table a foreign key points to is written beside it. A dashed red arrow is a composite foreign key: these are what make the database refuse a car in a two-wheeler bay and a booking on someone else's vehicle.

SmartPark: relational schema (17 relations, arrows run from a foreign key to the table it references)

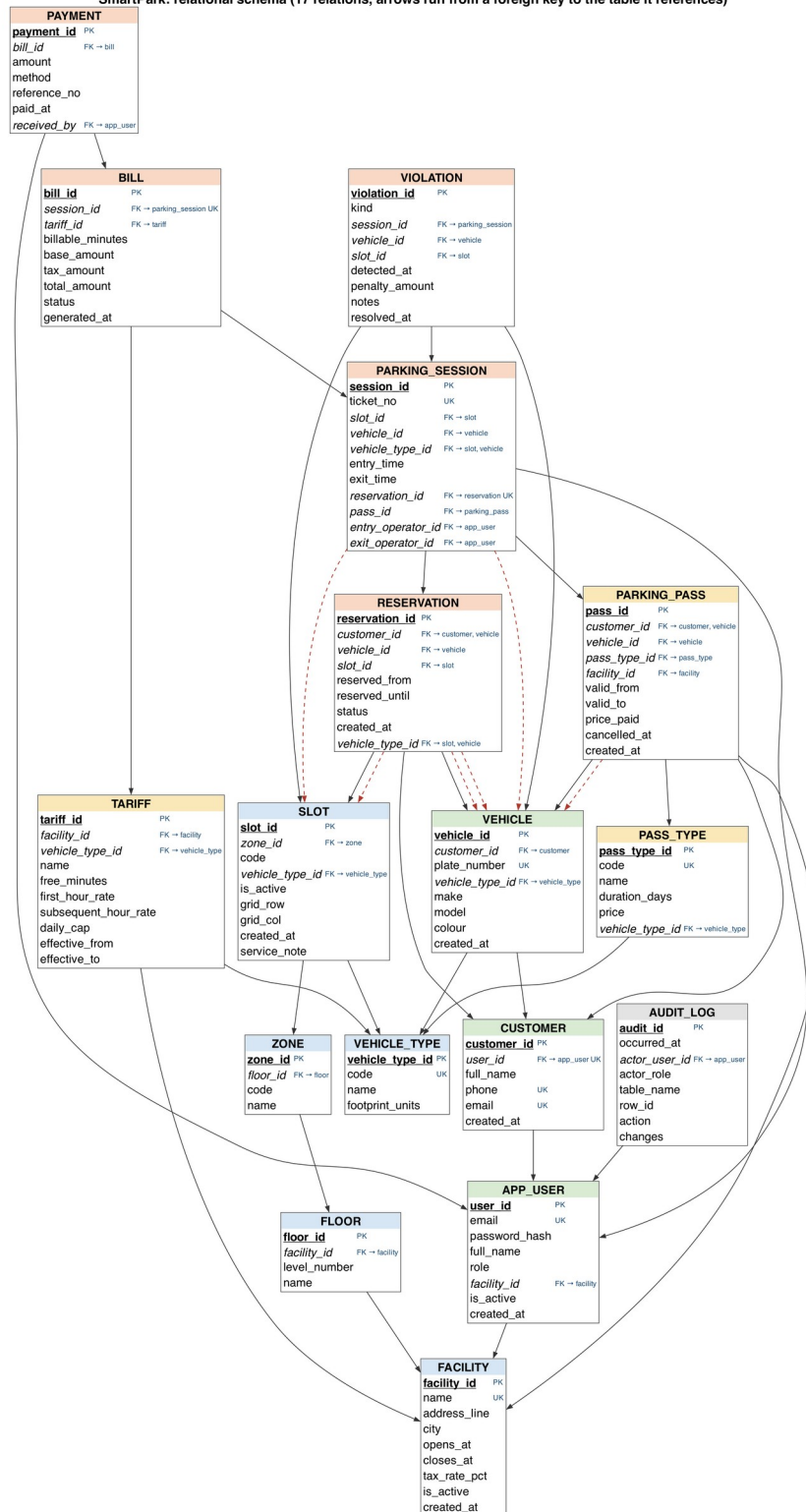


Figure 9: Relational schema: the 17 relations and their foreign keys (underlined: primary key; italic: foreign key; UK: unique)

Foreign keys

All 36 foreign keys. A key marked required cannot be NULL; the ON DELETE column says what happens to the child rows when a parent row is deleted (RESTRICT refuses the deletion, CASCADE deletes the children with it, SET NULL keeps them and clears the link).

Table 7: Foreign keys of the relational schema

Child table	Foreign key	Parent table	Parent key	Required	ON DELETE
app_user	facility_id	facility	facility_id	no	RESTRICT
audit_log	actor_user_id	app_user	user_id	no	SET NULL
bill	session_id	parking_session	session_id	yes	CASCADE
bill	tariff_id	tariff	tariff_id	yes	RESTRICT
customer	user_id	app_user	user_id	no	SET NULL
floor	facility_id	facility	facility_id	yes	CASCADE
parking_pass	customer_id	customer	customer_id	yes	RESTRICT
parking_pass	facility_id	facility	facility_id	yes	CASCADE
parking_pass	pass_type_id	pass_type	pass_type_id	yes	RESTRICT
parking_pass	vehicle_id	vehicle	vehicle_id	yes	RESTRICT
parking_pass	vehicle_id, customer_id	vehicle	vehicle_id, customer_id	yes	NO ACTION
parking_session	entry_operator_id	app_user	user_id	no	SET NULL
parking_session	exit_operator_id	app_user	user_id	no	SET NULL
parking_session	pass_id	parking_pass	pass_id	no	SET NULL
parking_session	reservation_id	reservation	reservation_id	no	SET NULL
parking_session	slot_id, vehicle_type_id	slot	slot_id, vehicle_type_id	yes	RESTRICT
parking_session	vehicle_id, vehicle_type_id	vehicle	vehicle_id, vehicle_type_id	yes	RESTRICT
pass_type	vehicle_type_id	vehicle_type	vehicle_type_id	yes	RESTRICT
payment	bill_id	bill	bill_id	yes	CASCADE
payment	received_by	app_user	user_id	no	SET NULL
reservation	customer_id	customer	customer_id	yes	RESTRICT
reservation	slot_id	slot	slot_id	yes	RESTRICT
reservation	slot_id, vehicle_type_id	slot	slot_id, vehicle_type_id	yes	NO ACTION
reservation	vehicle_id	vehicle	vehicle_id	yes	RESTRICT
reservation	vehicle_id, customer_id	vehicle	vehicle_id, customer_id	yes	NO ACTION
reservation	vehicle_id, vehicle_type_id	vehicle	vehicle_id, vehicle_type_id	yes	NO ACTION
slot	vehicle_type_id	vehicle_type	vehicle_type_id	yes	RESTRICT
slot	zone_id	zone	zone_id	yes	CASCADE
tariff	facility_id	facility	facility_id	yes	CASCADE
tariff	vehicle_type_id	vehicle_type	vehicle_type_id	yes	RESTRICT

Child table	Foreign key	Parent table	Parent key	Require d	ON DELETE
vehicle	customer_id	customer	customer_id	yes	RESTRICT
vehicle	vehicle_type_id	vehicle_type	vehicle_type_id	yes	RESTRICT
violation	session_id	parking_session	session_id	no	SET NULL
violation	slot_id	slot	slot_id	no	SET NULL
violation	vehicle_id	vehicle	vehicle_id	yes	RESTRICT
zone	floor_id	floor	floor_id	yes	CASCADE

Order of creation

A table can only be created after every table its foreign keys point to. This gives eight levels, and the migration scripts follow this order.

Table 8: Levels at which the tables are created

Level	Tables
1	facility, vehicle_type
2	app_user, floor, pass_type, tariff
3	audit_log, customer, zone
4	slot, vehicle
5	parking_pass, reservation
6	parking_session
7	bill, violation
8	payment

5.2 Functional Dependencies

The functional dependencies of every relation, generated from the implemented schema and written determinant -> dependents. Each primary key determines all other attributes of its table; each candidate key (a UNIQUE, NOT NULL attribute set) determines the primary key; a UNIQUE column that may be NULL is marked as such. Composite UNIQUE constraints that contain the primary key, such as (slot_id, vehicle_type_id), exist only as foreign-key targets and are not listed because they add no dependency.

```
-- facility
F1  facility_id      -> name, address_line, city, opens_at, closes_at, tax_rate_pct,
    is_active, created_at
F2  name             -> facility_id (candidate key)

-- floor
F3  floor_id         -> facility_id, level_number, name
F4  {facility_id, level_number}-> floor_id (candidate key)

-- zone
F5  zone_id          -> floor_id, code, name
F6  {floor_id, code} -> zone_id (candidate key)

-- vehicle_type
F7  vehicle_type_id  -> code, name, footprint_units
F8  code             -> vehicle_type_id (candidate key)
```

```
-- slot
F9  slot_id      -> zone_id, code, vehicle_type_id, is_active, grid_row, grid_col,
                  created_at, service_note
F10 {zone_id, code} -> slot_id      (candidate key)

-- app_user
F11 user_id      -> email, password_hash, full_name, role, facility_id, is_active,
                  created_at
F12 email        -> user_id      (candidate key)

-- customer
F13 customer_id  -> user_id, full_name, phone, email, created_at
F14 user_id      -> customer_id  (unique when present; nullable, so not a candidate key)
F15 phone        -> customer_id  (candidate key)
F16 email        -> customer_id  (unique when present; nullable, so not a candidate key)

-- vehicle
F17 vehicle_id   -> customer_id, plate_number, vehicle_type_id, make, model, colour,
                  created_at
F18 plate_number -> vehicle_id   (candidate key)

-- tariff
F19 tariff_id    -> facility_id, vehicle_type_id, name, free_minutes, first_hour_rate,
                  subsequent_hour_rate, daily_cap, effective_from, effective_to

-- pass_type
F20 pass_type_id -> code, name, duration_days, price, vehicle_type_id
F21 code         -> pass_type_id (candidate key)

-- parking_pass
F22 pass_id      -> customer_id, vehicle_id, pass_type_id, facility_id, valid_from,
                  valid_to, price_paid, cancelled_at, created_at

-- reservation
F23 reservation_id -> customer_id, vehicle_id, slot_id, reserved_from, reserved_until,
                  status, created_at, vehicle_type_id

-- parking_session
F24 session_id    -> ticket_no, slot_id, vehicle_id, vehicle_type_id, entry_time,
                  exit_time, reservation_id, pass_id, entry_operator_id, exit_operator_id
F25 ticket_no     -> session_id  (candidate key)
F26 reservation_id -> session_id  (unique when present; nullable, so not a candidate key)

-- bill
F27 bill_id       -> session_id, tariff_id, billable_minutes, base_amount, tax_amount,
                  total_amount, status, generated_at
F28 session_id    -> bill_id     (candidate key)

-- payment
F29 payment_id    -> bill_id, amount, method, reference_no, paid_at, received_by

-- violation
F30 violation_id  -> kind, session_id, vehicle_id, slot_id, detected_at, penalty_amount,
                  notes, resolved_at

-- audit_log
F31 audit_id      -> occurred_at, actor_user_id, actor_role, table_name, row_id, action,
                  changes

-- business rule 1 (partial unique indexes on open sessions)
F32 slot_id      WHERE exit_time IS NULL -> session_id  (business rule 1)
F33 vehicle_id   WHERE exit_time IS NULL -> session_id  (business rule 1)

Derived values, deliberately not stored as independent data:
D1 {base_amount, tax_amount} -> total_amount      (GENERATED ALWAYS column)
D2 session_id -> base_amount                    (fn_calculate_charge, enforced by trigger)
D3 exit_time IS NULL -> session is active        (no status column on parking_session)
D4 {valid_from, valid_to, cancelled_at, now()} -> pass state
D5 {open sessions, live reservations} -> slot state (no status column on slot)
```

5.3 Normalization to Third Normal Form

The design was derived from a single flat parking record of the kind a manual ticket book holds: ticket number, plate, owner name and phone, vehicle type, facility, floor, zone, bay, entry and exit times, the tariff rates, the charge and a list of payments.

First Normal Form (1NF)

1NF requires atomic values and no repeating groups. The flat record breaks it twice: one ticket can carry several payments (a repeating group), and a tariff is a time series of price versions. Payments were moved to their own relation **PAYMENT** (**payment_id**, **bill_id**, **amount**, **method**, ...), and tariffs to **TARIFF** with an **effective_from**/**effective_to** period per version. Every column now holds one value of one type.

Second Normal Form (2NF)

2NF removes partial dependencies on part of a composite key. Using the natural composite key {facility, floor level, zone code, bay code} for a bay, the facility address depends on the facility alone and the floor name on {facility, level} alone. Decomposing into **FACILITY**, **FLOOR**, **ZONE** and **SLOT**, each with a single-column surrogate key and the natural key kept as a UNIQUE constraint (e.g. UNIQUE (zone_id, code)), leaves every non-key attribute depending on the whole key.

Third Normal Form (3NF)

3NF removes transitive dependencies through non-key attributes. Three were present: (a) session → vehicle → owner name and phone, resolved by **CUSTOMER** and **VEHICLE**; (b) vehicle → vehicle type → type name, resolved by the **VEHICLE_TYPE** lookup; (c) reservation → customer → contact details, resolved by referencing **customer_id** only. Derived values (bill total, slot status, session status, pass status) are computed rather than stored, so they cannot disagree with their sources.

Controlled denormalization

Three columns look redundant but are deliberate and cannot drift:

`parking_session.vehicle_type_id` and `reservation.vehicle_type_id` exist only so the composite foreign keys can force slot type and vehicle type to be equal (Section 4.3); both keys pin the value, so it cannot disagree with either source.

`parking_pass.price_paid` records what the customer actually paid, which must stay fixed when the shelf price in `pass_type` changes, so it is a different fact rather than a copy.

5.4 Result

Table 9: Normal form of every table

Tables	1NF	2NF	3NF	Note
facility, floor, zone	✓	✓	✓	natural keys kept as UNIQUE constraints
slot	✓	✓	✓	no stored status; derived from sessions and reservations
vehicle_type, pass_type	✓	✓	✓	lookups that remove transitive dependencies
app_user, customer, vehicle	✓	✓	✓	
tariff	✓	✓	✓	versioned; resolves the 1NF time series
parking_pass	✓	✓	✓	price_paid is a distinct historical fact
reservation, parking_session	✓	✓	✓	vehicle_type_id pinned by two composite FKs
bill	✓	✓	✓	total_amount GENERATED, not stored twice
payment	✓	✓	✓	resolves the 1NF repeating group
violation, audit_log	✓	✓	✓	audit changes held as one JSONB value

Every table is also in **BCNF**: in each relation the only determinants of non-trivial dependencies are candidate keys.

6. Data Dictionary

Generated from the live PostgreSQL catalogue (pg_attribute, pg_constraint), so it describes the implemented schema exactly. *Null* shows whether the column accepts NULL; *Keys* lists primary, foreign and unique keys; CHECK and EXCLUDE constraints follow each table.

6.1 facility

A parking building. Owns floors, tariffs and operators.

Table 10: Structure of facility (5 rows in the sample database)

Column	Data type	Null	Keys	Default
facility_id	bigint	No	PK	identity
name	text	No	UNIQUE	
address_line	text	No		
city	text	No		
opens_at	time	No		'00:00:00'::time without t...
closes_at	time	No		'23:59:00'::time without t...
tax_rate_pct	numeric(5,2)	No		18.00
is_active	boolean	No		true
created_at	timestampz	No		now()

Constraints:

ck_facility_name_not_blank: CHECK ((length(btrim(name)) > 0))
ck_facility_tax_rate: CHECK (((tax_rate_pct >= (0)) AND (tax_rate_pct <= (100))))

6.2 floor

A level within a facility. level_number 0 is ground, negatives are basements.

Table 11: Structure of floor (5 rows in the sample database)

Column	Data type	Null	Keys	Default
floor_id	bigint	No	PK	identity
facility_id	bigint	No	FK → facility, UNIQUE (composite)	
level_number	integer	No	UNIQUE (composite)	
name	text	No		

Constraints:

ck_floor_level_range: CHECK (((level_number >= '-5'::integer) AND (level_number <= 50)))

6.3 zone

A block of slots on one floor, e.g. Zone A. Used for aisle grouping in the slot map.

Table 12: Structure of zone (10 rows in the sample database)

Column	Data type	Null	Keys	Default
zone_id	bigint	No	PK	identity
floor_id	bigint	No	FK → floor, UNIQUE (composite)	
code	text	No	UNIQUE (composite)	
name	text	No		

Constraints:

ck_zone_code_shape: CHECK ((code ~ '^[A-Z]{1,3}\$'))

6.4 vehicle_type

Vehicle categories. Referenced by slot, vehicle, tariff and pass_type.

Table 13: Structure of vehicle_type (5 rows in the sample database)

Column	Data type	Null	Keys	Default
vehicle_type_id	bigint	No	PK	identity
code	text	No	UNIQUE	
name	text	No		
footprint_units	smallint	No		1

Constraints:

ck_vehicle_type_code_shape: CHECK ((code ~ '^[A-Z]{2,10}\$'))

ck_vehicle_type_footprint: CHECK (((footprint_units >= 1) AND (footprint_units <= 4)))

6.5 slot

One parking space. Occupancy is NOT stored here - it is derived from parking_session via v_current_occupancy.

Table 14: Structure of slot (134 rows in the sample database)

Column	Data type	Null	Keys	Default
slot_id	bigint	No	PK, UNIQUE (composite)	identity
zone_id	bigint	No	FK → zone, UNIQUE (composite)	
code	text	No	UNIQUE (composite)	
vehicle_type_id	bigint	No	FK → vehicle_type, UNIQUE (composite)	

Column	Data type	Null	Keys	Default
is_active	boolean	No		true
grid_row	smallint	Yes		
grid_col	smallint	Yes		
created_at	timestampz	No		now()
service_note	text	Yes		

Constraints:

ck_slot_code_shape: CHECK ((code ~ '^[A-Z0-9-]{2,16}\$'))
ck_slot_note_only_when_out: CHECK (((service_note IS NULL) OR ((NOT is_active) AND (char_length(service_note) >= 1) AND (char_length(service_note) <= 200))))

6.6 app_user

Login identity and role. Drives every row-level security policy.

Table 15: Structure of *app_user* (8 rows in the sample database)

Column	Data type	Null	Keys	Default
user_id	bigint	No	PK	identity
email	citext	No	UNIQUE	
password_hash	text	No		
full_name	text	No		
role	user_role	No		'customer':user_role
facility_id	bigint	Yes	FK → facility	
is_active	boolean	No		true
created_at	timestampz	No		now()

Constraints:

ck_app_user_email_shape: CHECK ((email ~ '^[^@[:space:]]+@[^@[:space:]]+\.[^@[:space:]]+\$'::citext))
ck_app_user_name_not_blank: CHECK ((length(btrim(full_name)) > 0))
ck_app_user_operator_has_facility: CHECK (((role = 'operator') = (facility_id IS NOT NULL)))

6.7 customer

A parking customer. user_id is NULL for walk-ins recorded at the gate.

Table 16: Structure of *customer* (31 rows in the sample database)

Column	Data type	Null	Keys	Default
customer_id	bigint	No	PK	identity
user_id	bigint	Yes	FK → app_user, UNIQUE	

Column	Data type	Null	Keys	Default
full_name	text	No		
phone	text	No	UNIQUE	
email	citext	Yes	UNIQUE	
created_at	timestampz	No		now()

Constraints:

ck_customer_email_shape: CHECK (((email IS NULL) OR (email ~ '^^[^[:space:]]+@[^[:space:]]+\.[^[:space:]]+\$'::citext)))
ck_customer_name_not_blank: CHECK ((length(btrim(full_name)) > 0))
ck_customer_phone_shape: CHECK ((phone ~ '^[0-9]{10}\$'))

6.8 vehicle

A customer vehicle, plate_number is UNIQUE and is the operator search key.

Table 17: Structure of vehicle (46 rows in the sample database)

Column	Data type	Null	Keys	Default
vehicle_id	bigint	No	PK, UNIQUE (composite)	identity
customer_id	bigint	No	FK → customer, UNIQUE (composite)	
plate_number	text	No	UNIQUE	
vehicle_type_id	bigint	No	FK → vehicle_type, UNIQUE (composite)	
make	text	Yes		
model	text	Yes		
colour	text	Yes		
created_at	timestampz	No		now()

Constraints:

ck_vehicle_plate_shape: CHECK ((plate_number ~ '^[A-Z]{2}[0-9]{1,2}[A-Z]{1,3}[0-9]{4}\$'))

6.9 tariff

Versioned price list. Superseded rows are closed with effective_to so historical bills stay reproducible.

Table 18: Structure of tariff (11 rows in the sample database)

Column	Data type	Null	Keys	Default
tariff_id	bigint	No	PK	identity
facility_id	bigint	No	FK → facility	
vehicle_type_id	bigint	No	FK → vehicle_type	
name	text	No		

Column	Data type	Null	Keys	Default
free_minutes	integer	No		15
first_hour_rate	numeric(10,2)	No		
subsequent_hour_rate	numeric(10,2)	No		
daily_cap	numeric(10,2)	No		
effective_from	timestampz	No		now()
effective_to	timestampz	Yes		

Constraints:

ck_tariff_cap_sane: CHECK ((daily_cap >= first_hour_rate))
ck_tariff_free_minutes: CHECK (((free_minutes >= 0) AND (free_minutes <= 1440)))
ck_tariff_rates_non_negative: CHECK (((first_hour_rate >= (0)) AND (subsequent_hour_rate >= (0)) AND (daily_cap >= (0))))
ck_tariff_window_valid: CHECK (((effective_to IS NULL) OR (effective_to > effective_from)))
ex_tariff_no_overlap: EXCLUDE USING gist (facility_id WITH =, vehicle_type_id WITH =, tstzrange(effective_from, effective_to) WITH &&)

6.10 pass_type

Sellable pass products. A pass is an instance of a pass_type bought by a customer.

Table 19: Structure of pass_type (15 rows in the sample database)

Column	Data type	Null	Keys	Default
pass_type_id	bigint	No	PK	identity
code	text	No	UNIQUE	
name	text	No		
duration_days	integer	No		
price	numeric(10,2)	No		
vehicle_type_id	bigint	No	FK → vehicle_type	

Constraints:

ck_pass_type_duration: CHECK (((duration_days >= 1) AND (duration_days <= 366)))
ck_pass_type_price: CHECK ((price >= (0)))

6.11 parking_pass

A purchased pass. Active-ness is derived from the date window and cancelled_at, not stored.

Table 20: Structure of parking_pass (27 rows in the sample database)

Column	Data type	Null	Keys	Default
pass_id	bigint	No	PK	identity
customer_id	bigint	No	FK → customer, FK → vehicle (composite)	

Column	Data type	Null	Keys	Default
vehicle_id	bigint	No	FK → vehicle, FK → vehicle (composite)	
pass_type_id	bigint	No	FK → pass_type	
facility_id	bigint	No	FK → facility	
valid_from	timestampz	No		
valid_to	timestampz	No		
price_paid	numeric(10,2)	No		
cancelled_at	timestampz	Yes		
created_at	timestampz	No		now()

Constraints:

ck_pass_price: CHECK ((price_paid >= (0)))
ck_pass_window: CHECK ((valid_to > valid_from))
ex_pass_no_overlap: EXCLUDE USING gist (vehicle_id WITH =, facility_id WITH =, tstzrange(valid_from, valid_to) WITH &&) WHERE ((cancelled_at IS NULL))

6.12 reservation

A slot held for a future arrival. Stale holds are expired by fn_expire_stale_reservations.

Table 21: Structure of reservation (32 rows in the sample database)

Column	Data type	Null	Keys	Default
reservation_id	bigint	No	PK	identity
customer_id	bigint	No	FK → customer, FK → vehicle (composite)	
vehicle_id	bigint	No	FK → vehicle, FK → vehicle (composite)	
slot_id	bigint	No	FK → slot, FK → slot (composite)	
reserved_from	timestampz	No		
reserved_until	timestampz	No		
status	reservation_status	No		'held'
created_at	timestampz	No		now()
vehicle_type_id	bigint	No	FK → slot (composite), FK → vehicle (composite)	

Constraints:

ck_reservation_window: CHECK ((reserved_until > reserved_from))
ex_reservation_no_overlap: EXCLUDE USING gist (slot_id WITH =, tstzrange(reserved_from, reserved_until) WITH &&) WHERE ((status = ANY (ARRAY['held', 'confirmed'])))

6.13 parking_session

A vehicle occupying a slot. Active when exit_time IS NULL; there is deliberately no status column.

Table 22: Structure of parking_session (1629 rows in the sample database)

Column	Data type	Null	Keys	Default
session_id	bigint	No	PK	identity
ticket_no	text	No	UNIQUE	
slot_id	bigint	No	FK → slot (composite)	
vehicle_id	bigint	No	FK → vehicle (composite)	
vehicle_type_id	bigint	No	FK → slot (composite), FK → vehicle (composite)	
entry_time	timestampz	No		now()
exit_time	timestampz	Yes		
reservation_id	bigint	Yes	FK → reservation, UNIQUE	
pass_id	bigint	Yes	FK → parking_pass	
entry_operator_id	bigint	Yes	FK → app_user	
exit_operator_id	bigint	Yes	FK → app_user	

Constraints:

ck_session_exit_after_entry: CHECK (((exit_time IS NULL) OR (exit_time > entry_time)))
ck_session_ticket_shape: CHECK ((ticket_no ~ '^TK-[0-9A-Z]{6,12}\$'))

6.14 bill

One bill per completed session. base_amount is overwritten from fn_calculate_charge by trigger; total_amount is generated.

Table 23: Structure of bill (1611 rows in the sample database)

Column	Data type	Null	Keys	Default
bill_id	bigint	No	PK	identity
session_id	bigint	No	FK → parking_session, UNIQUE	
tariff_id	bigint	No	FK → tariff	
billable_minutes	integer	No		
base_amount	numeric(10,2)	No		
tax_amount	numeric(10,2)	No		
total_amount	numeric(10,2)	Yes		generated
status	bill_status	No		'unpaid'
generated_at	timestampz	No		now()

Constraints:

ck_bill_base_non_negative: CHECK ((base_amount >= (0)))
ck_bill_minutes_non_negative: CHECK ((billable_minutes >= 0))
ck_bill_tax_non_negative: CHECK ((tax_amount >= (0)))

6.15 payment

A recorded receipt against a bill. Several payments may settle one bill (partly_paid). No payment credentials are stored.

Table 24: Structure of payment (1398 rows in the sample database)

Column	Data type	Null	Keys	Default
payment_id	bigint	No	PK	identity
bill_id	bigint	No	FK → bill	
amount	numeric(10,2)	No		
method	payment_method	No		
reference_no	text	Yes		
paid_at	timestampz	No		now()
received_by	bigint	Yes	FK → app_user	

Constraints:

ck_payment_amount_positive: CHECK ((amount > (0)))

6.16 violation

Logged infringements: overstay, wrong slot type, no valid pass, unpaid exit, reservation no-show.

Table 25: Structure of violation (26 rows in the sample database)

Column	Data type	Null	Keys	Default
violation_id	bigint	No	PK	identity
kind	violation_type	No		
session_id	bigint	Yes	FK → parking_session	
vehicle_id	bigint	No	FK → vehicle	
slot_id	bigint	Yes	FK → slot	
detected_at	timestampz	No		now()
penalty_amount	numeric(10,2)	No		0
notes	text	Yes		
resolved_at	timestampz	Yes		

Constraints:

ck_violation_penalty: CHECK ((penalty_amount >= (0)))

ck_violation_resolved_after: CHECK (((resolved_at IS NULL) OR (resolved_at >= detected_at)))

6.17 audit_log

Append-only change history written by trigger. changes holds the full row for INSERT/DELETE and {column: {from, to}} for UPDATE.

Table 26: Structure of audit_log (11 rows in the sample database)

Column	Data type	Null	Keys	Default
audit_id	bigint	No	PK	identity
occurred_at	timestampz	No		now()
actor_user_id	bigint	Yes	FK → app_user	
actor_role	user_role	Yes		
table_name	text	No		
row_id	bigint	No		
action	text	No		
changes	jsonb	No		

Constraints:

ck_audit_action: CHECK ((action = ANY (ARRAY['INSERT', 'UPDATE', 'DELETE'])))

7. SQL Commands Used (DDL and DML) with Sample Outputs

The database is built by 19 numbered migration scripts in db/migrations/, applied in order by setup.sh. This section shows representative statements; every output is the real text printed by psql.

7.1 Data Definition Language (DDL)

Creating the tables

```
CREATE TABLE customer
CREATE TABLE IF NOT EXISTS customer (
  customer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id     BIGINT     UNIQUE,
  full_name   TEXT       NOT NULL,
  phone       TEXT       NOT NULL UNIQUE,
  email       CITEXT      UNIQUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),

  CONSTRAINT fk_customer_user
    FOREIGN KEY (user_id) REFERENCES app_user (user_id)
    ON UPDATE CASCADE ON DELETE SET NULL,
  CONSTRAINT ck_customer_phone_shape CHECK (phone ~ '^[0-9]{10}$'),
  CONSTRAINT ck_customer_name_not_blank CHECK (length(btrim(full_name)) > 0)
);
```

```
CREATE TABLE slot
CREATE TABLE IF NOT EXISTS slot (
  slot_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  zone_id      BIGINT NOT NULL,
  code         TEXT   NOT NULL,
  vehicle_type_id BIGINT NOT NULL,
  is_active    BOOLEAN NOT NULL DEFAULT TRUE,
  grid_row     SMALLINT,
  grid_col     SMALLINT,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),

  CONSTRAINT fk_slot_zone
    FOREIGN KEY (zone_id) REFERENCES zone (zone_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT fk_slot_vehicle_type
    FOREIGN KEY (vehicle_type_id) REFERENCES vehicle_type (vehicle_type_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT uq_slot_zone_code UNIQUE (zone_id, code),
  CONSTRAINT ck_slot_code_shape CHECK (code ~ '^[A-Z0-9-]{2,16}$'),
  CONSTRAINT uq_slot_id_vehicle_type UNIQUE (slot_id, vehicle_type_id)
);
```

CREATE TABLE vehicle

```
CREATE TABLE IF NOT EXISTS vehicle (
  vehicle_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id     BIGINT NOT NULL,
  plate_number    TEXT    NOT NULL UNIQUE,
  vehicle_type_id BIGINT NOT NULL,
  make            TEXT,
  model           TEXT,
  colour          TEXT,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),

  CONSTRAINT fk_vehicle_customer
    FOREIGN KEY (customer_id) REFERENCES customer (customer_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_vehicle_vehicle_type
    FOREIGN KEY (vehicle_type_id) REFERENCES vehicle_type (vehicle_type_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,

  CONSTRAINT ck_vehicle_plate_shape
    CHECK (plate_number ~ '^([A-Z]{2}[0-9]{1,2}([A-Z]{1,3}[0-9]{4})$)'),

  CONSTRAINT uq_vehicle_id_vehicle_type UNIQUE (vehicle_id, vehicle_type_id)
);
```

CREATE TABLE parking_session

```
CREATE TABLE IF NOT EXISTS parking_session (
  session_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  ticket_no       TEXT    NOT NULL UNIQUE,
  slot_id         BIGINT NOT NULL,
  vehicle_id      BIGINT NOT NULL,
  vehicle_type_id BIGINT NOT NULL,
  entry_time      TIMESTAMPTZ NOT NULL DEFAULT now(),
  exit_time       TIMESTAMPTZ,
  reservation_id  BIGINT UNIQUE,
  pass_id         BIGINT,
  entry_operator_id BIGINT,
  exit_operator_id BIGINT,

  CONSTRAINT fk_session_reservation
    FOREIGN KEY (reservation_id) REFERENCES reservation (reservation_id)
    ON UPDATE CASCADE ON DELETE SET NULL,
  CONSTRAINT fk_session_pass
    FOREIGN KEY (pass_id) REFERENCES parking_pass (pass_id)
    ON UPDATE CASCADE ON DELETE SET NULL,
  CONSTRAINT fk_session_entry_operator
    FOREIGN KEY (entry_operator_id) REFERENCES app_user (user_id)
    ON UPDATE CASCADE ON DELETE SET NULL,
  CONSTRAINT fk_session_exit_operator
    FOREIGN KEY (exit_operator_id) REFERENCES app_user (user_id)
    ON UPDATE CASCADE ON DELETE SET NULL,

  CONSTRAINT fk_session_slot_type_match
    FOREIGN KEY (slot_id, vehicle_type_id)
    REFERENCES slot (slot_id, vehicle_type_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_session_vehicle_type_match
    FOREIGN KEY (vehicle_id, vehicle_type_id)
    REFERENCES vehicle (vehicle_id, vehicle_type_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,

  CONSTRAINT ck_session_exit_after_entry
    CHECK (exit_time IS NULL OR exit_time > entry_time),

  CONSTRAINT ck_session_ticket_shape CHECK (ticket_no ~ '^TK-[0-9A-Z]{6,12}$')
);
```

CREATE TABLE bill

```
CREATE TABLE IF NOT EXISTS bill (
  bill_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  session_id       BIGINT NOT NULL UNIQUE,
  tariff_id        BIGINT NOT NULL,
  billable_minutes INTEGER NOT NULL,
  base_amount      NUMERIC(10,2) NOT NULL,
  tax_amount       NUMERIC(10,2) NOT NULL,
  total_amount     NUMERIC(10,2)
                  GENERATED ALWAYS AS (base_amount + tax_amount) STORED,
  status           bill_status NOT NULL DEFAULT 'unpaid',
  generated_at     TIMESTAMPTZ NOT NULL DEFAULT now(),

  CONSTRAINT fk_bill_session
    FOREIGN KEY (session_id) REFERENCES parking_session (session_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT fk_bill_tariff
    FOREIGN KEY (tariff_id) REFERENCES tariff (tariff_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,

  CONSTRAINT ck_bill_base_non_negative CHECK (base_amount >= 0),
  CONSTRAINT ck_bill_tax_non_negative  CHECK (tax_amount >= 0),
  CONSTRAINT ck_bill_minutes_non_negative CHECK (billable_minutes >= 0)
);
```

CREATE TABLE payment

```
CREATE TABLE IF NOT EXISTS payment (
  payment_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  bill_id     BIGINT NOT NULL,
  amount      NUMERIC(10,2) NOT NULL,
  method      payment_method NOT NULL,
  reference_no TEXT,
  paid_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  received_by BIGINT,

  CONSTRAINT fk_payment_bill
    FOREIGN KEY (bill_id) REFERENCES bill (bill_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT fk_payment_received_by
    FOREIGN KEY (received_by) REFERENCES app_user (user_id)
    ON UPDATE CASCADE ON DELETE SET NULL,

  CONSTRAINT ck_payment_amount_positive CHECK (amount > 0)
);
```

Output (for each statement)

```
CREATE TABLE
```

Constraints that are not simple column checks

Partial unique indexes and an exclusion constraint

```
CREATE UNIQUE INDEX IF NOT EXISTS uq_active_session_slot
  ON parking_session (slot_id)
  WHERE exit_time IS NULL;

CREATE UNIQUE INDEX IF NOT EXISTS uq_active_session_vehicle
  ON parking_session (vehicle_id)
  WHERE exit_time IS NULL;

-- Overlapping live reservations for one slot are impossible (from reservation):
CONSTRAINT ex_reservation_no_overlap EXCLUDE USING gist (
  slot_id WITH =, tstzrange(reserved_from, reserved_until) WITH &&
) WHERE (status IN ('held', 'confirmed'))
```

Indexes on frequently searched columns

```

From 008_indexes.sql
CREATE INDEX ix_vehicle_plate_upper ON vehicle (upper(plate_number));
CREATE INDEX ix_session_entry_time ON parking_session (entry_time DESC);
CREATE INDEX ix_session_open_by_slot ON parking_session (slot_id) WHERE exit_time IS NULL;
CREATE INDEX ix_session_open_by_vehicle ON parking_session (vehicle_id) WHERE exit_time IS NULL;
CREATE INDEX ix_reservation_status_until ON reservation (status, reserved_until);
CREATE INDEX ix_reservation_slot_window ON reservation USING gist (slot_id, tstzrange(reserved_from, reserved_until)) WHERE status IN ('held', 'confirmed');
CREATE INDEX ix_bill_status_generated ON bill (status, generated_at DESC);
CREATE INDEX ix_payment_bill ON payment (bill_id);
CREATE INDEX ix_payment_paid_at ON payment (paid_at DESC);
CREATE INDEX ix_slot_zone ON slot (zone_id);
CREATE INDEX ix_slot_type ON slot (vehicle_type_id);
CREATE INDEX ix_zone_floor ON zone (floor_id);
CREATE INDEX ix_floor_facility ON floor (facility_id);
CREATE INDEX ix_pass_vehicle_window ON parking_pass (vehicle_id, valid_from, valid_to) WHERE cancelled_at IS NULL;
CREATE INDEX ix_violation_detected ON violation (detected_at DESC);
CREATE INDEX ix_violation_unresolved ON violation (kind, detected_at DESC) WHERE resolved_at IS NULL;
CREATE INDEX ix_customer_user ON customer (user_id);
CREATE INDEX ix_vehicle_customer ON vehicle (customer_id);

```

A view

```

CREATE VIEW v_peak_hours
CREATE OR REPLACE VIEW v_peak_hours
WITH (security_invoker = true) AS
SELECT
    fl.facility_id,
    EXTRACT(HOUR FROM ps.entry_time)::INTEGER AS hour_of_day,
    COUNT(*) AS entries,
    COUNT(*) FILTER (WHERE ps.exit_time IS NULL) AS still_parked,
    ROUND(AVG(EXTRACT(EPOCH FROM (COALESCE(ps.exit_time, now()) - ps.entry_time)) / 60.0), 1)
        AS avg_stay_minutes,
    RANK() OVER (PARTITION BY fl.facility_id ORDER BY COUNT(*) DESC) AS busyness_rank
FROM parking_session ps
JOIN slot s ON s.slot_id = ps.slot_id
JOIN zone z ON z.zone_id = s.zone_id
JOIN floor fl ON fl.floor_id = z.floor_id
GROUP BY fl.facility_id, EXTRACT(HOUR FROM ps.entry_time);

```

7.2 Data Manipulation Language (DML)

INSERT, UPDATE and DELETE on customer and vehicle, run inside a transaction that is rolled back so the sample data is unchanged. RETURNING shows each affected row.

```

psql session (statements and their output)
BEGIN;
BEGIN
INSERT INTO customer (full_name, phone, email)
VALUES ('Report Demo', '9000012345', 'report.demo@example.com')
RETURNING customer_id, full_name, phone;
 customer_id | full_name | phone
-----+-----+-----
          37 | Report Demo | 9000012345
(1 row)

INSERT 0 1
INSERT INTO vehicle (customer_id, vehicle_type_id, plate_number, make, model, colour)
SELECT customer_id, 2, 'TS09RD2026', 'Hyundai', 'i20', 'White' FROM customer WHERE phone = '9000012345'
RETURNING vehicle_id, plate_number, vehicle_type_id;
 vehicle_id | plate_number | vehicle_type_id
-----+-----+-----
          50 | TS09RD2026 | 2
(1 row)

INSERT 0 1
UPDATE vehicle SET colour = 'Silver' WHERE plate_number = 'TS09RD2026'
RETURNING plate_number, colour;
 plate_number | colour
-----+-----
 TS09RD2026 | Silver
(1 row)

```

```

UPDATE 1
DELETE FROM vehicle WHERE plate_number = 'TS09RD2026' RETURNING vehicle_id, plate_number;
  vehicle_id | plate_number
-----+-----
          50 | TS09RD2026
(1 row)

DELETE 1
DELETE FROM customer WHERE phone = '9000012345' RETURNING customer_id, full_name;
  customer_id | full_name
-----+-----
          37 | Report Demo
(1 row)

DELETE 1
ROLLBACK;
ROLLBACK

```

7.3 A Transaction: Gate Entry and Exit

Arrival and departure are single calls to stored functions, each of which is one transaction. `fn_gate_entry` finds and locks a free bay of the right type and opens the session; `fn_gate_exit` closes it and raises the bill, whose amounts the database computes from the tariff.

```

psql session
BEGIN;
BEGIN
SELECT * FROM fn_gate_entry('TS10WX1515', 1, 2);
  session_id | ticket_no | slot_id | slot_code | pass_id
-----+-----+-----+-----+-----
        1633 | TK-C9D0862D |      1 | B-A-01 | 
(1 row)

UPDATE parking_session SET entry_time = now() - interval '3 hours 20 minutes'
WHERE vehicle_id = (SELECT vehicle_id FROM vehicle WHERE plate_number = 'TS10WX1515') AND exit_time IS NULL;
UPDATE 1
SELECT session_id, bill_id, billable_minutes, base_amount, tax_amount, total_amount, slot_code
FROM fn_gate_exit('TS10WX1515', 2);
  session_id | bill_id | billable_minutes | base_amount | tax_amount | total_amount | slot_code
-----+-----+-----+-----+-----+-----+-----
        1633 |      1615 |             200 |        35.00 |         6.30 |         41.30 | B-A-01
(1 row)

ROLLBACK;
ROLLBACK

```

7.4 Sample Data: Rows in Every Table

Every table holds at least five records. Seed scripts `010_seed.sql` and `011_seed_history.sql` generate a month of realistic activity with morning and evening peaks.

Row counts

```
SELECT table_name, row_count FROM <every base table>;
```

table_name	row_count
app_user	8
audit_log	11
bill	1611
customer	31
facility	5
floor	5
parking_pass	27
parking_session	1629
pass_type	15
payment	1398
reservation	32
slot	134
tariff	11
vehicle	46
vehicle_type	5
violation	26
zone	10
(17 rows)	

8. Queries with Outputs

Ten queries covering joins, outer joins, aggregation with GROUP BY and HAVING, correlated and NOT EXISTS subqueries, window functions, views, set operations and a stored function. The full set of 18 demonstration queries is in docs/database/queries.sql.

Q1. INNER JOIN — open sessions with vehicle, owner and bay

```
SELECT ps.ticket_no, v.plate_number, c.full_name, s.code AS bay,
       to_char(ps.entry_time, 'DD Mon HH24:MI') AS entered
FROM parking_session ps
JOIN vehicle v ON v.vehicle_id = ps.vehicle_id
JOIN customer c ON c.customer_id = v.customer_id
JOIN slot s ON s.slot_id = ps.slot_id
WHERE ps.exit_time IS NULL
ORDER BY ps.entry_time DESC
LIMIT 5;
```

Output

ticket_no	plate_number	full_name	bay	entered
TK-CF3C3969	TS130P5050	Farhan Ali	L1-A-05	05 Oct 18:36
TK-A938042C	AP16EF8899	Kavya Subramanian	L1-A-02	05 Oct 18:19
TK-6366717A	TS07GH2121	Suresh Menon	G-A-04	05 Oct 18:09
TK-A957760A	AP39AB1717	Fatima Qureshi	G-A-06	05 Oct 17:58
TK-BE471335	TS07MN8888	Vikram Reddy	G-C-14	05 Oct 17:40

(5 rows)

Q2. LEFT OUTER JOIN — customers and how many vehicles they own (including none)

```
SELECT c.full_name, COUNT(v.vehicle_id) AS vehicles
FROM customer c
LEFT JOIN vehicle v ON v.customer_id = c.customer_id
GROUP BY c.customer_id, c.full_name
ORDER BY vehicles, c.full_name
LIMIT 5;
```

Output

full_name	vehicles
Anil Varma	0
Joseph Mathew	0
Rekha Chandran	0
Farhan Ali	1
Gaurav Sinha	1

(5 rows)

Q3. Aggregate with GROUP BY / HAVING — revenue per facility and vehicle type (last 30 days)

```
SELECT f.name AS facility, vt.name AS vehicle_type,
       COUNT(*) AS bills, SUM(b.total_amount) AS revenue,
       ROUND(AVG(b.total_amount), 2) AS avg_bill
FROM bill b
JOIN parking_session ps ON ps.session_id = b.session_id
JOIN slot s ON s.slot_id = ps.slot_id
JOIN zone z ON z.zone_id = s.zone_id
JOIN floor fl ON fl.floor_id = z.floor_id
JOIN facility f ON f.facility_id = fl.facility_id
JOIN vehicle_type vt ON vt.vehicle_type_id = ps.vehicle_type_id
WHERE b.generated_at > now() - interval '30 days'
GROUP BY f.name, vt.name
HAVING SUM(b.total_amount) > 0
ORDER BY revenue DESC
LIMIT 6;
```

Output

facility	vehicle_type	bills	revenue	avg_bill
SmartPark Central	Hatchback / Sedan	501	57312.60	114.40
SmartPark Central	SUV / MUV	190	28237.40	148.62
SmartPark Central	Light Commercial	101	24526.30	242.83
SmartPark Riverside	Hatchback / Sedan	180	21481.90	119.34
SmartPark Central	Electric Vehicle	152	14295.70	94.05
SmartPark Riverside	SUV / MUV	86	11080.20	128.84

(6 rows)

Q4. Correlated subquery — vehicles whose longest stay exceeds their own average by 2x

```
SELECT v.plate_number,
       (SELECT MAX(EXTRACT(EPOCH FROM (ps.exit_time - ps.entry_time)) / 3600)::numeric(6,1)
        FROM parking_session ps WHERE ps.vehicle_id = v.vehicle_id) AS longest_hours,
       (SELECT AVG(EXTRACT(EPOCH FROM (ps.exit_time - ps.entry_time)) / 3600)::numeric(6,1)
        FROM parking_session ps WHERE ps.vehicle_id = v.vehicle_id) AS avg_hours
FROM vehicle v
WHERE (SELECT MAX(ps.exit_time - ps.entry_time) FROM parking_session ps WHERE ps.vehicle_id = v.vehicle_id)
      > 2 * (SELECT AVG(ps.exit_time - ps.entry_time) FROM parking_session ps WHERE ps.vehicle_id = v.vehicle_id)
ORDER BY longest_hours DESC
LIMIT 5;
```

Output

plate_number	longest_hours	avg_hours
KA51YZ2233	39.6	4.4
TS13EF1919	39.1	3.5
TS07GH2121	38.9	4.4
TS10CD6677	36.3	5.0
TS11CD1818	36.1	4.0

(5 rows)

Q5. NOT EXISTS — free car bays at SmartPark Central right now

```
SELECT s.code, vt.name AS built_for
  FROM slot s
 JOIN zone z ON z.zone_id = s.zone_id
 JOIN floor fl ON fl.floor_id = z.floor_id
 JOIN vehicle_type vt ON vt.vehicle_type_id = s.vehicle_type_id
 WHERE fl.facility_id = 1 AND s.is_active AND vt.code = 'CAR'
    AND NOT EXISTS (SELECT 1 FROM parking_session ps
                     WHERE ps.slot_id = s.slot_id AND ps.exit_time IS NULL)
 ORDER BY s.code
 LIMIT 5;
```

Output

code	built_for
B-A-04	Hatchback / Sedan
B-A-06	Hatchback / Sedan
B-A-07	Hatchback / Sedan
B-A-08	Hatchback / Sedan
B-A-09	Hatchback / Sedan

(5 rows)

Q6. Window function — daily revenue with a running total

```
SELECT d::date AS day, revenue,
       SUM(revenue) OVER (ORDER BY d) AS running_total,
       RANK() OVER (ORDER BY revenue DESC) AS rank_by_revenue
  FROM (SELECT date_trunc('day', generated_at) AS d, SUM(total_amount) AS revenue
        FROM bill WHERE generated_at > now() - interval '7 days'
        GROUP BY 1) t
 ORDER BY d;
```

Output

day	revenue	running_total	rank_by_revenue
2026-09-28	879.10	879.10	7
2026-09-29	6436.90	7316.00	4
2026-09-30	5846.90	13162.90	5
2026-10-01	6572.60	19735.50	2
2026-10-02	7262.90	26998.40	1
2026-10-03	3551.80	30550.20	6
2026-10-04	6501.80	37052.00	3
2026-10-05	578.20	37630.20	8

(8 rows)

Q7. View — peak arrival hours (v_peak_hours)

```
SELECT hour_of_day, entries, avg_stay_minutes, busyness_rank
  FROM v_peak_hours
 WHERE facility_id = 1
 ORDER BY busyness_rank
 LIMIT 5;
```

Output

hour_of_day	entries	avg_stay_minutes	busyness_rank
9	122	222.2	1
17	117	197.4	2
10	107	187.6	3
16	101	233.0	4
18	98	194.5	5

(5 rows)

Q8. View — current occupancy by zone (v_current_occupancy)

```
SELECT facility_name, zone_code, COUNT(*) AS bays,
       COUNT(*) FILTER (WHERE slot_state = 'occupied') AS occupied,
       COUNT(*) FILTER (WHERE slot_state = 'free') AS free
FROM v_current_occupancy
GROUP BY facility_name, zone_code
ORDER BY facility_name, zone_code
LIMIT 6;
```

Output

facility_name	zone_code	bays	occupied	free
SmartPark Central	A	40	7	32
SmartPark Central	B	40	4	33
SmartPark Central	C	14	2	11
SmartPark Riverside	A	26	4	22
SmartPark Riverside	B	14	1	13

(5 rows)

Q9. UNION ALL — combined revenue from parking bills and pass sales (30 days)

```
SELECT 'Parking bills' AS source, COUNT(*) AS transactions, SUM(total_amount) AS revenue
FROM bill WHERE generated_at > now() - interval '30 days'
UNION ALL
SELECT 'Pass sales', COUNT(*), SUM(price_paid)
FROM parking_pass WHERE created_at > now() - interval '30 days';
```

Output

source	transactions	revenue
Parking bills	1576	179849.70
Pass sales	15	34150.00

(2 rows)

Q10. Function — the charge computed by the database, checked against the tariff

```
SELECT b.bill_id, b.billable_minutes, t.free_minutes, t.first_hour_rate,
       t.subsequent_hour_rate, t.daily_cap, b.base_amount,
       fn_calculate_charge(b.session_id) AS recomputed
FROM bill b JOIN tariff t ON t.tariff_id = b.tariff_id
WHERE b.base_amount > 0
ORDER BY b.bill_id DESC
LIMIT 5;
```

Output								
bill_id	billable_minutes	free_minutes	first_hour_rate	subsequent_hour_rate	daily_cap	base_amount	recomputed	
1610	499	15	40.00	25.00	250.00	240.00	240.00	
1608	222	30	15.00	10.00	80.00	45.00	45.00	
1607	219	15	70.00	45.00	500.00	205.00	205.00	
1606	112	15	40.00	25.00	250.00	65.00	65.00	
1605	47	30	15.00	10.00	80.00	15.00	0.00	
(5 rows)								

9. UI Design and Screenshots

9.1 Design Approach

The interface is a browser application designed for a laptop at a parking control desk. A fixed sidebar groups the screens into Operations (Dashboard, Floor map, Gate, Reservations, Passes), Money (Billing, Reports) and Records (Customers, Settings). Bay states use colour, a word and an icon together, so no state relies on colour alone. Every list can be searched, sorted and exported to CSV; every action shows a confirmation or the database's reason for refusing it; destructive actions ask for confirmation first. Each screen reads and writes the PostgreSQL database through the API, so every change appears immediately in the database.

9.2 Screens

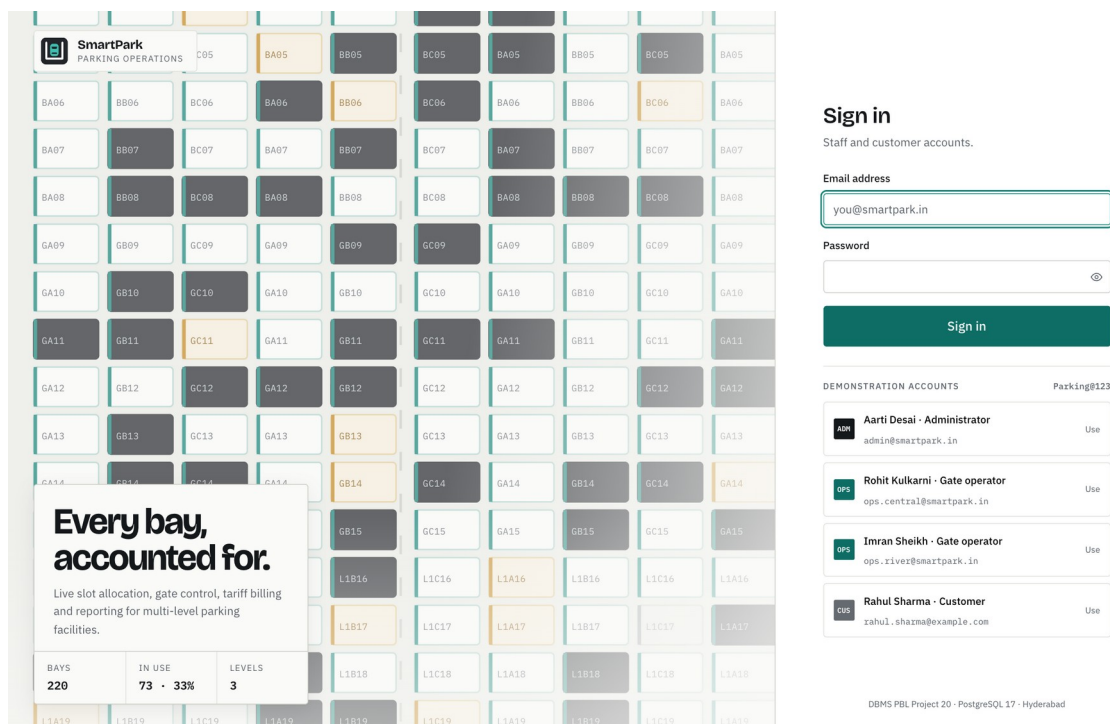


Figure 10: Sign-in screen (passwords are checked against bcrypt hashes)

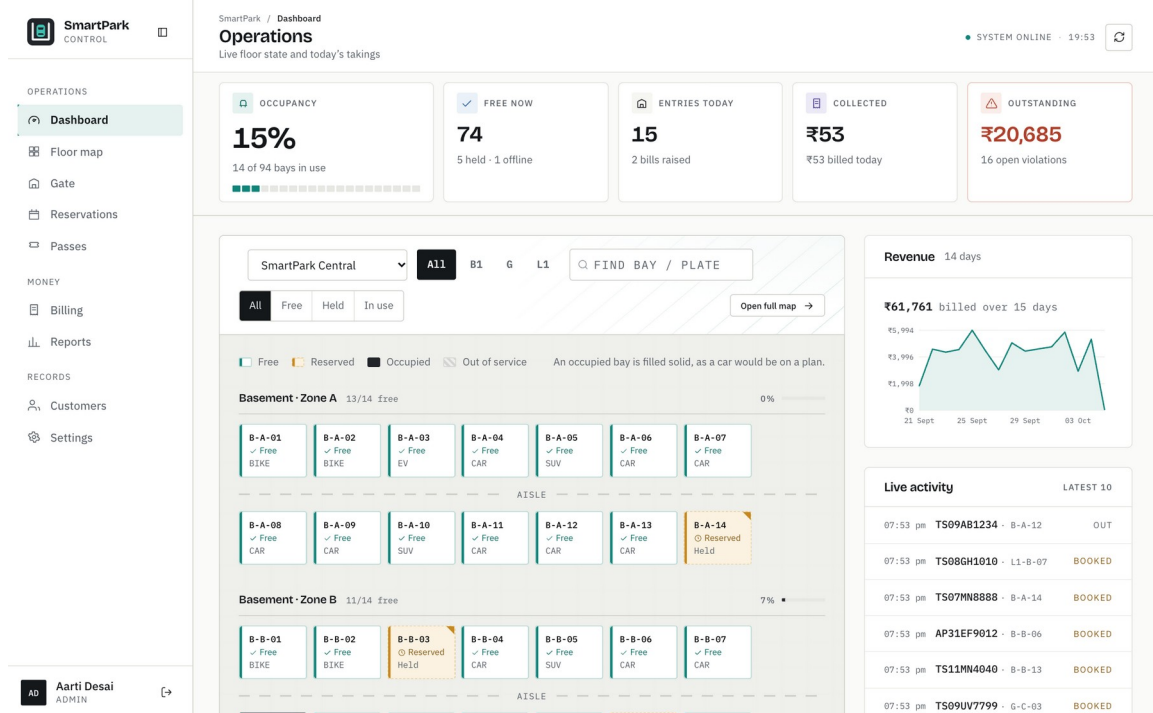


Figure 11: Dashboard: live occupancy, free bays, today's entries and takings, revenue trend and live activity

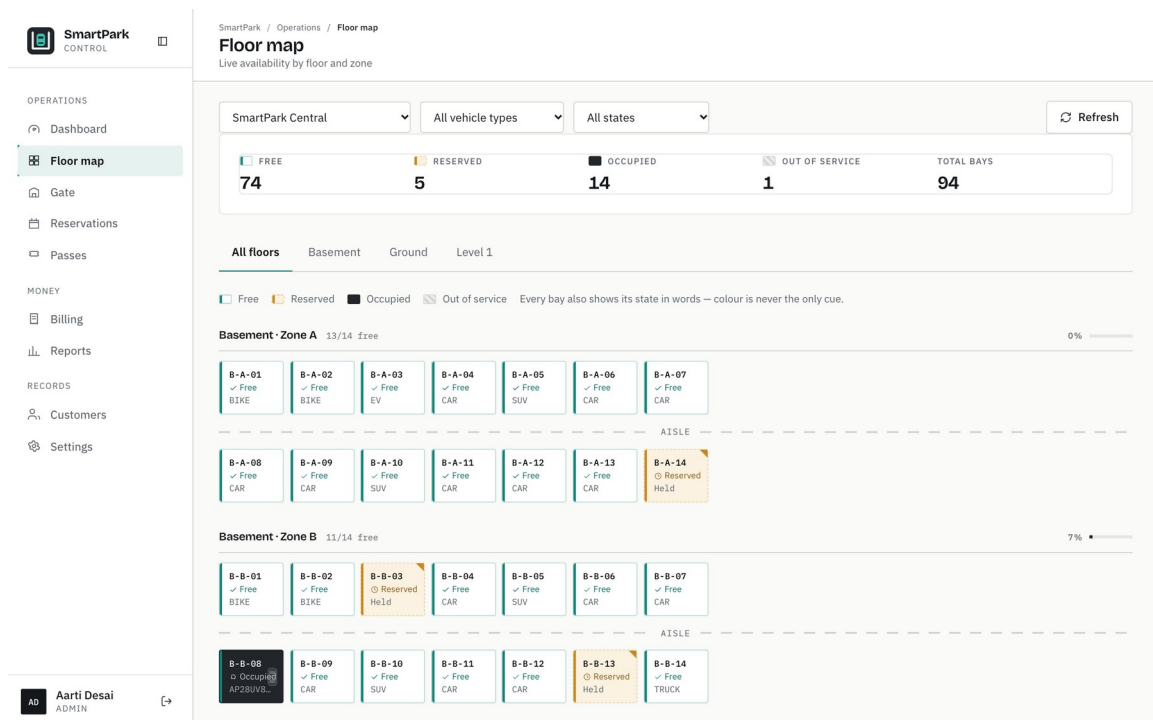


Figure 12: Floor map: every bay by floor and zone with its live state

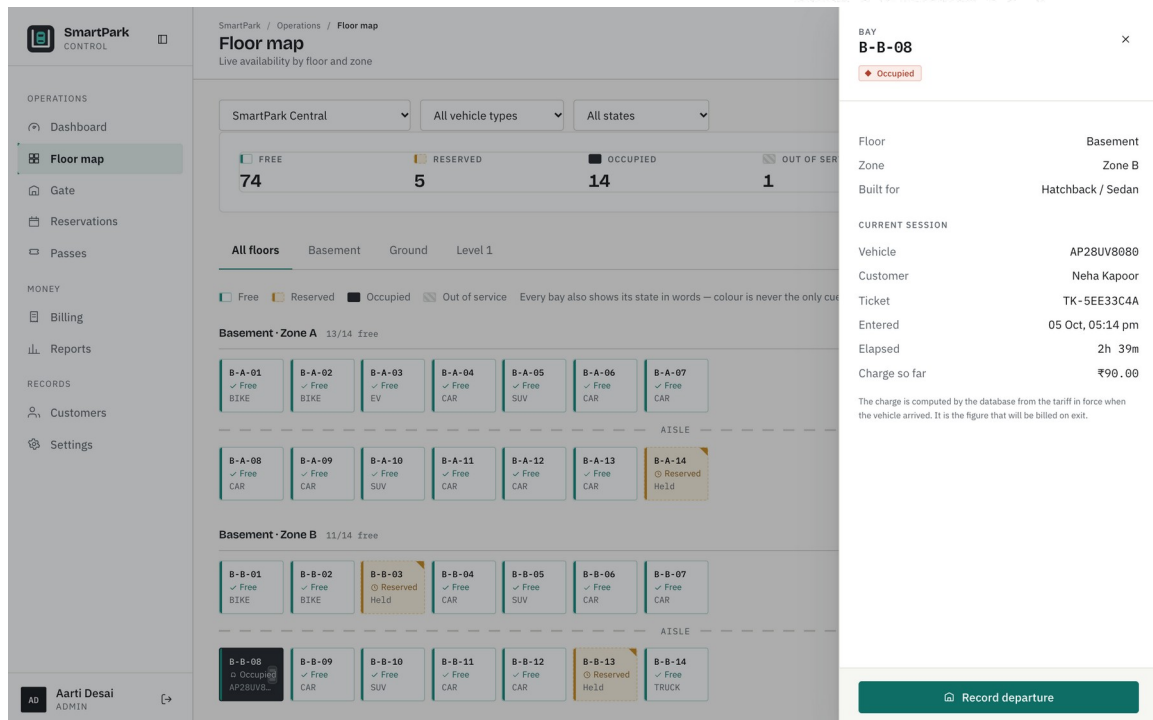


Figure 13: Bay details sheet, from which staff can take a bay out of service

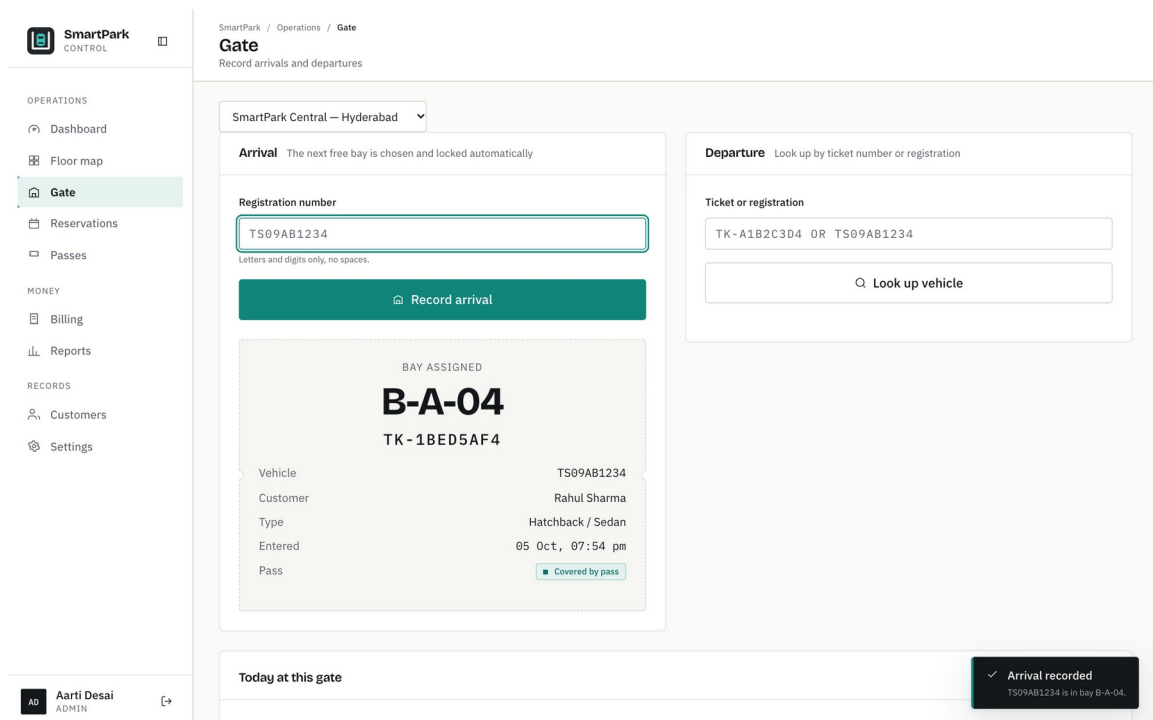
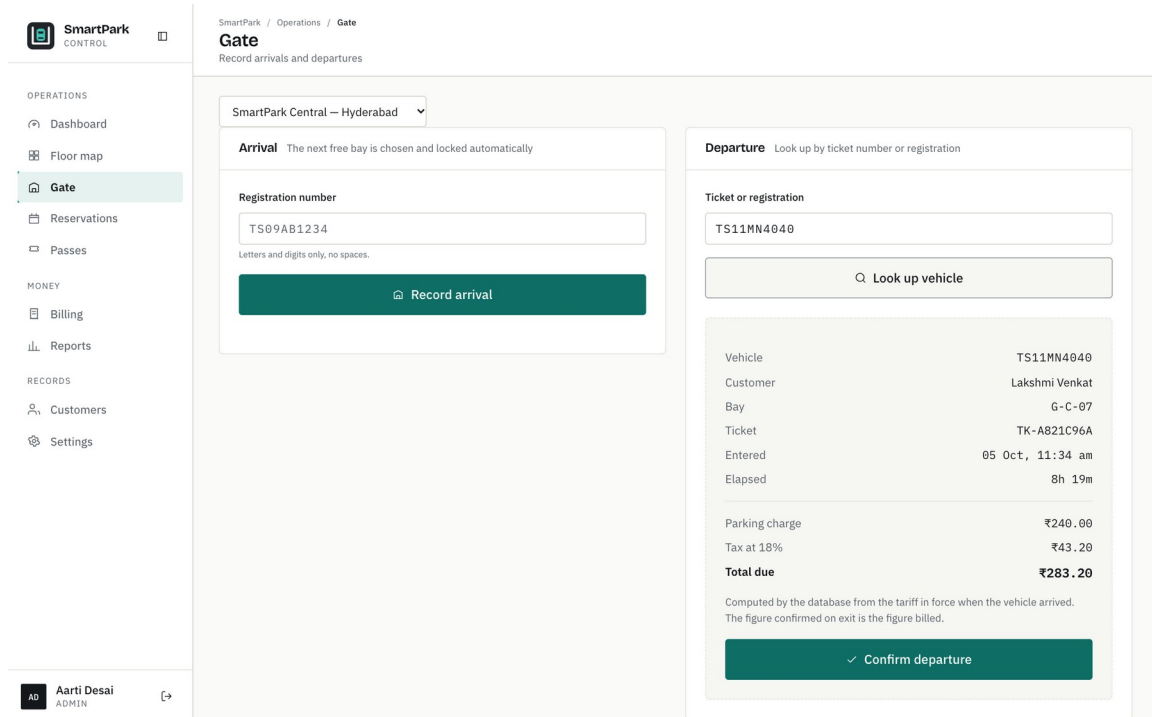


Figure 14: Gate arrival: the database chooses and locks a free bay and issues a ticket



SmartPark CONTROL

SmartPark / Operations / Gate

Gate
Record arrivals and departures

SmartPark Central — Hyderabad

Arrival The next free bay is chosen and locked automatically

Registration number
TS09AB1234
Letters and digits only, no spaces.

Record arrival

Departure Look up by ticket number or registration

Ticket or registration
TS11MN4040

Look up vehicle

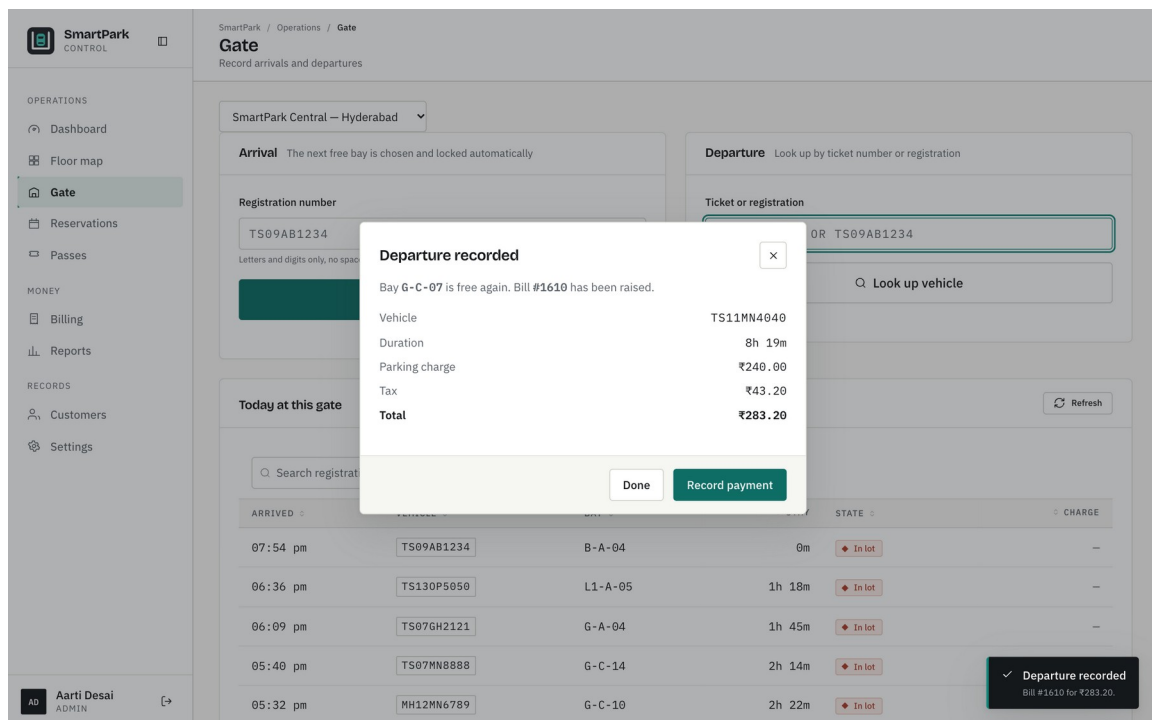
Vehicle	TS11MN4040
Customer	Lakshmi Venkat
Bay	G-C-07
Ticket	TK-A821C96A
Entered	05 Oct, 11:34 am
Elapsed	8h 19m

Parking charge	₹240.00
Tax at 18%	₹43.20
Total due	₹283.20

Computed by the database from the tariff in force when the vehicle arrived.
The figure confirmed on exit is the figure billed.

Confirm departure

Figure 15: Gate departure: the running charge computed by `fn_calculate_charge`



SmartPark CONTROL

SmartPark / Operations / Gate

Gate
Record arrivals and departures

SmartPark Central — Hyderabad

Arrival The next free bay is chosen and locked automatically

Registration number
TS09AB1234
Letters and digits only, no spaces.

Record arrival

Departure Look up by ticket number or registration

Ticket or registration
TS11MN4040 OR TS09AB1234

Look up vehicle

Departure recorded

Bay G-C-07 is free again. Bill #1610 has been raised.

Vehicle	TS11MN4040
Duration	8h 19m
Parking charge	₹240.00
Tax	₹43.20
Total	₹283.20

Record payment

Today at this gate

ARRIVED	TICKET	BAY	STATE	CHARGE
07:54 pm	TS09AB1234	B-A-04	0m	In lot
06:36 pm	TS130P5050	L1-A-05	1h 18m	In lot
06:09 pm	TS07GH2121	G-A-04	1h 45m	In lot
05:40 pm	TS07MN8888	G-C-14	2h 14m	In lot
05:32 pm	MH12MN6789	G-C-10	2h 22m	In lot

Departure recorded
Bill #1610 for ₹283.20.

Figure 16: Departure recorded: the bill raised by the database



SmartPark / Operations / Reservations

Reservations

Bays held for a vehicle over a booked window

OPERATIONS

Dashboard

Floor map

Gate

Reservations

Passes

MONEY

Billing

Reports

RECORDS

Customers

Settings

AD Aarti Desai ADMIN

Live 14 Fulfilled 6 Expired 6 Cancelled 6 All 32

+ New reservation

Two live holds can never overlap on one bay: the database refuses the second (ex_reservation_no_overlap). Lapsed holds expire automatically.

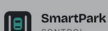
Search customer, registration or b

14 records

Export CSV

CUSTOMER	VEHICLE	BAY	FROM	UNTIL	STATUS
Rahul Sharma 9876543210	TS09EF4444 SUV / MUV	G-B-10 Ground	06 Oct, 08:23 am	06 Oct, 10:23 am	Confirmed
Nikhil Bhat 9925678901	KA51VZ2233 Hatchback / Sedan	B-A-11 Basement	06 Oct, 06:53 am	06 Oct, 08:53 am	Held
Sameer Joshi 9881234567	TS13QR3344 Light Commercial	L1-A-12 Level 1	06 Oct, 05:23 am	06 Oct, 07:23 am	Confirmed
Karan Malhotra 9867812345	MH12MN6789 SUV / MUV	G-B-05 Ground	06 Oct, 03:53 am	06 Oct, 05:53 am	Held
Neha Kapoor 9830123456	AP28UV8080 Hatchback / Sedan	B-B-13 Basement	06 Oct, 02:23 am	06 Oct, 04:23 am	Confirmed
Manish Tiwari 9969012345	TS09KL2424 Light Commercial	G-A-14 Ground	06 Oct, 12:53 am	06 Oct, 02:53 am	Held
Lakshmi Venkat 9992345012	TS11MN4040 Hatchback / Sedan	B-B-13 Basement	05 Oct, 11:23 pm	06 Oct, 01:23 am	Confirmed
Arjun Mehta 9823456781	AP31EF9012 Hatchback / Sedan	B-B-06 Basement	05 Oct, 09:53 pm	05 Oct, 11:53 pm	Held
Manish Tiwari	TS08GH1010	I 1-R-07			

Figure 17: Reservations with status filters



SmartPark / Operations / Reservations

Reservations

Bays held for a vehicle over a booked window

OPERATIONS

Dashboard

Floor map

Gate

Reservations

Passes

MONEY

Billing

Reports

RECORDS

Customers

Settings

AD Aarti Desai ADMIN

Live 14 Fulfilled 6 Expired 6 Cancelled 6 All 32

+ New reservation

Two live holds can never overlap on one bay: the database refuses the second (ex_reservation_no_overlap). Lapsed holds expire automatically.

Search customer

Export CSV

CUSTOMER	VEHICLE	BAY	FROM	UNTIL	STATUS
Rahul Sharma 9876543210					Confirmed
Nikhil Bhat 9925678901					Held
Sameer Joshi 9881234567					Confirmed
Karan Malhotra 9867812345					Held
Neha Kapoor 9830123456					Confirmed
Manish Tiwari 9969012345					Held
Lakshmi Venkat 9992345012	TS11MN4040 Hatchback / Sedan	B-B-13 Basement	05 Oct, 11:23 pm	06 Oct, 01:23 am	Confirmed
Arjun Mehta 9823456781	AP31EF9012 Hatchback / Sedan	B-B-06 Basement	05 Oct, 09:53 pm	05 Oct, 11:53 pm	Held
Manish Tiwari	TS08GH1010	I 1-R-07			

New reservation

Customer *

Choose a customer...

Vehicle *

Choose a customer first

From *

05/10/2026, 20:24

Until *

05/10/2026, 22:24

Bay *

Choose a vehicle first

Bays built for this vehicle's type and free now. A clash with another booking in your window is caught when you save.

Cancel

Create reservation

Figure 18: New reservation form

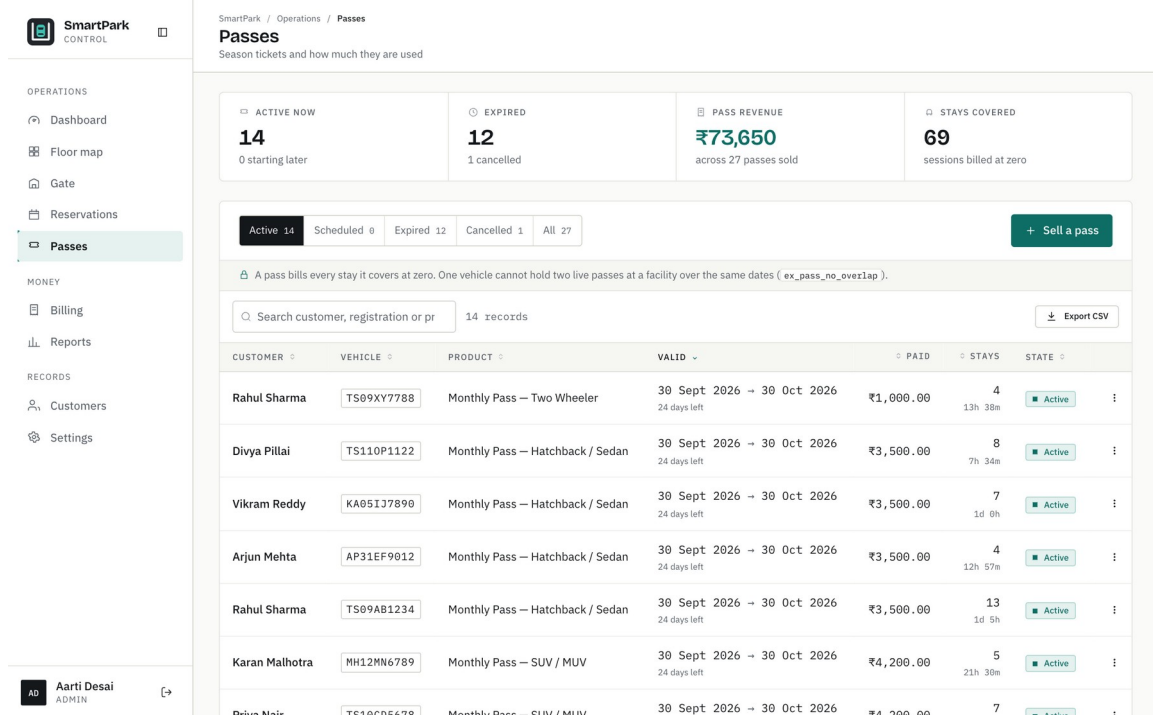


Figure 19: Parking passes

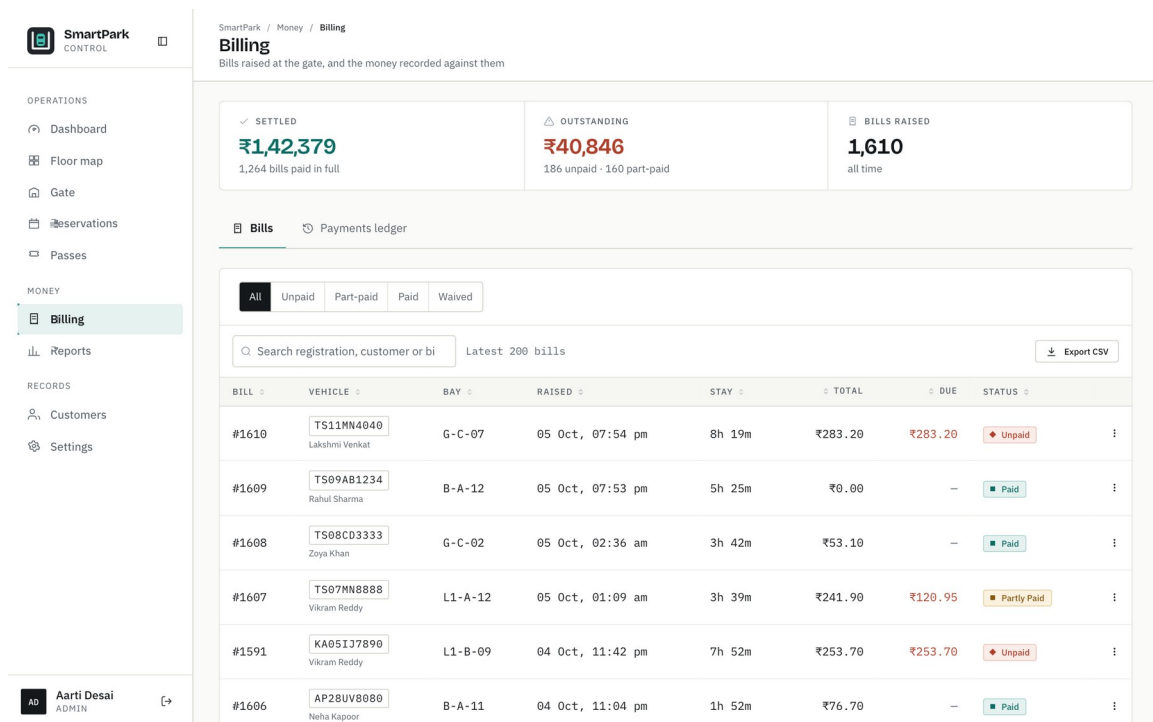


Figure 20: Billing: bills with status and amount due

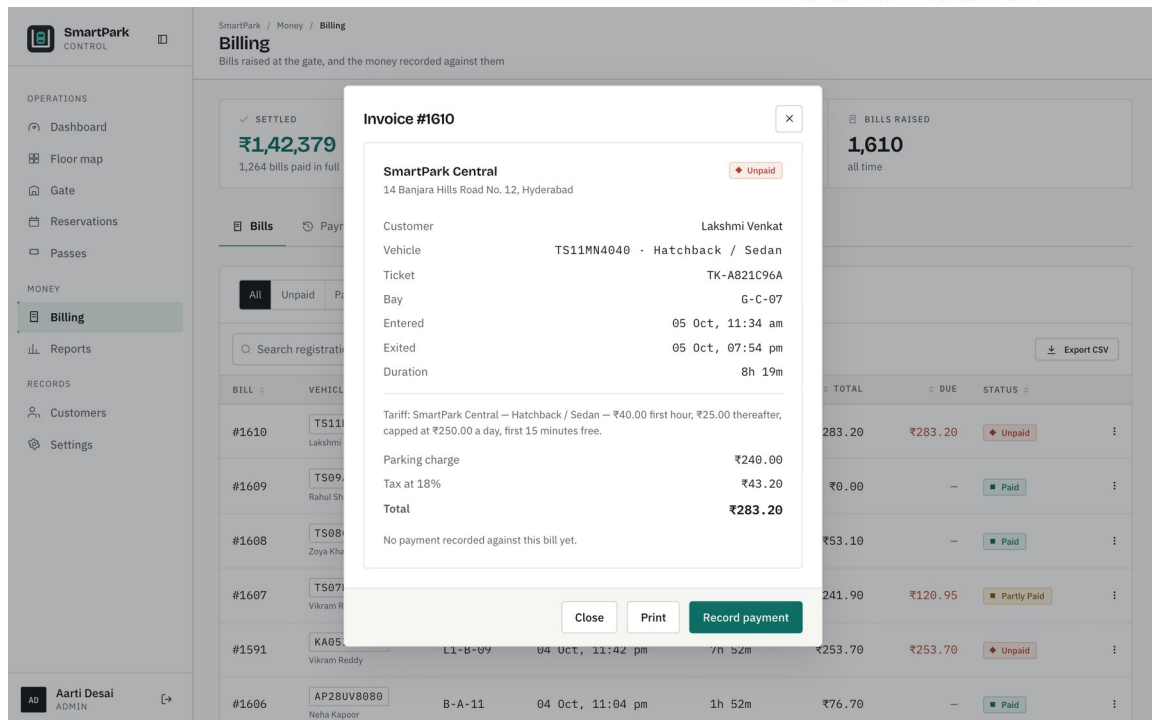


Figure 21: Invoice for a bill, showing the tariff that priced it

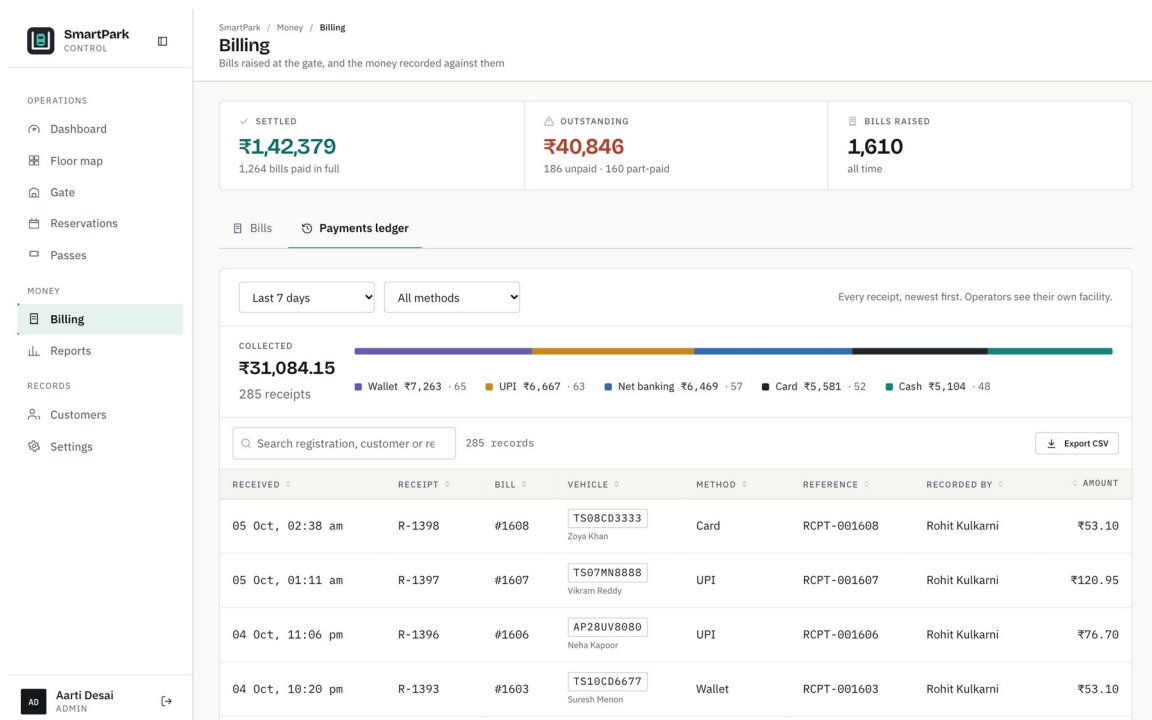


Figure 22: Payments ledger with the mix of payment methods

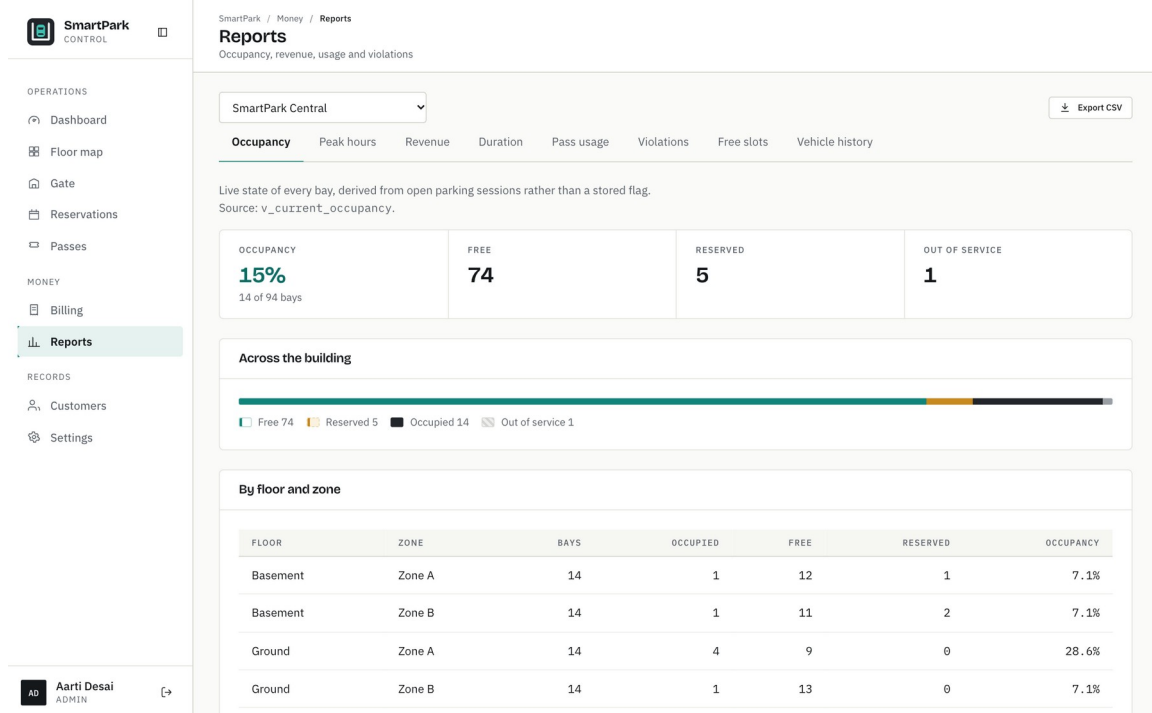


Figure 23: Report: occupancy (v_current_occupancy)

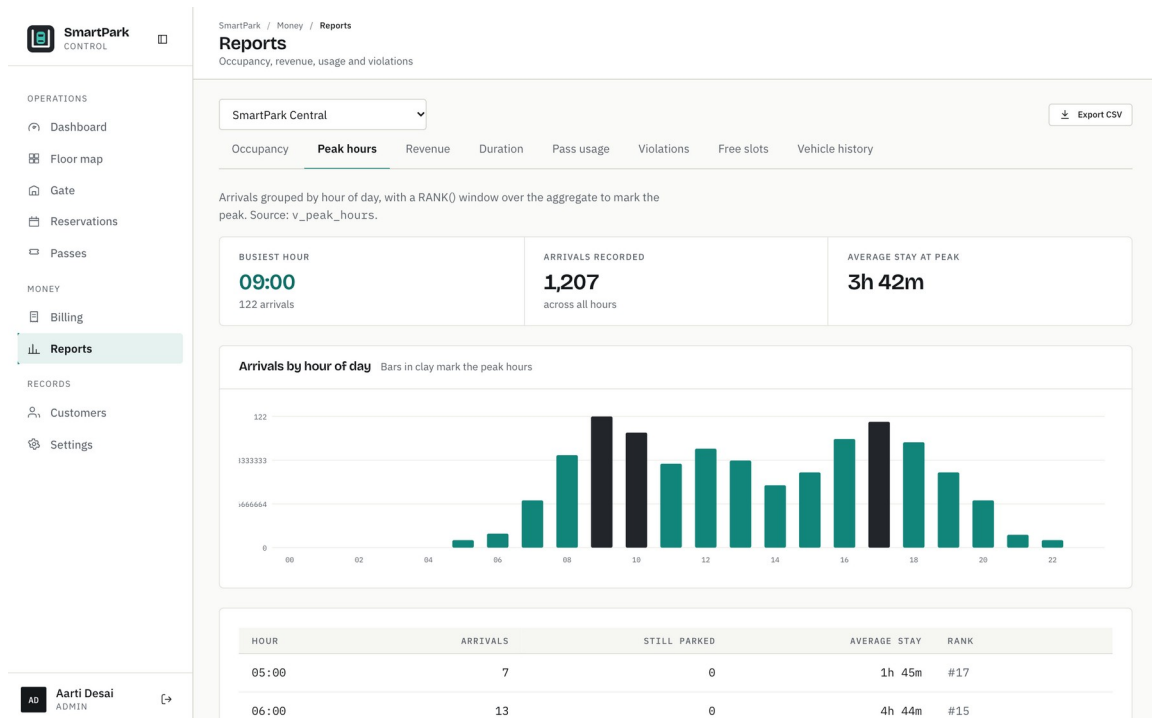


Figure 24: Report: peak hours (v_peak_hours)

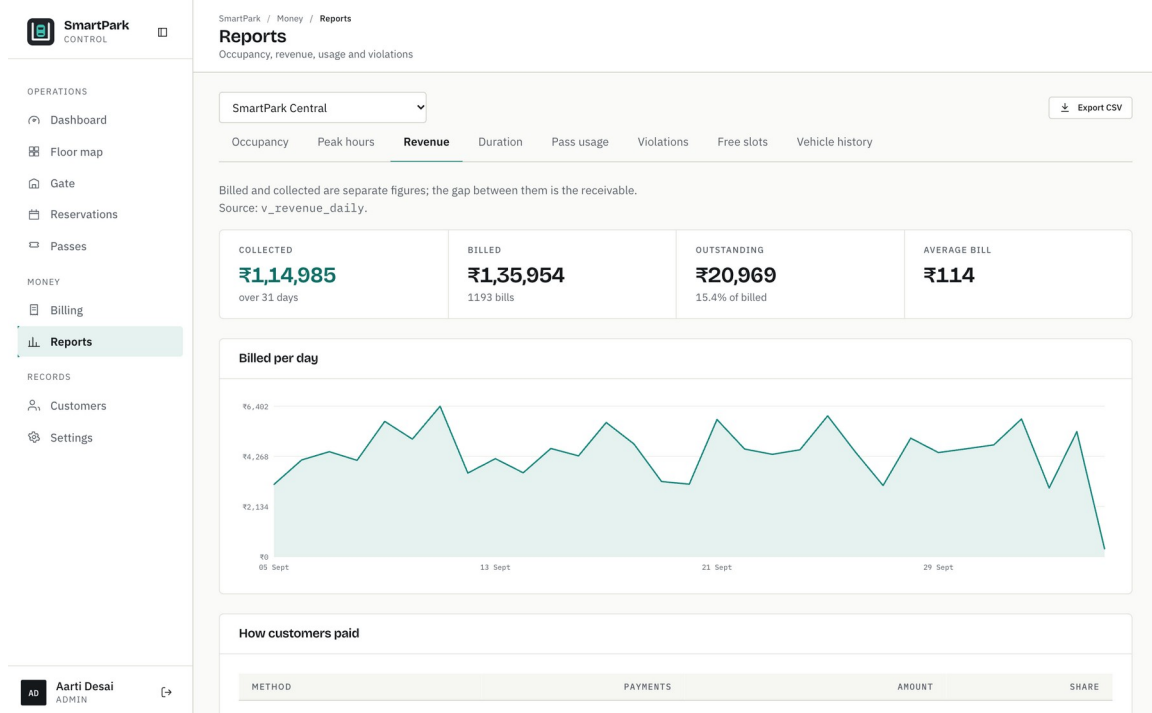


Figure 25: Report: revenue (v_revenue_daily)

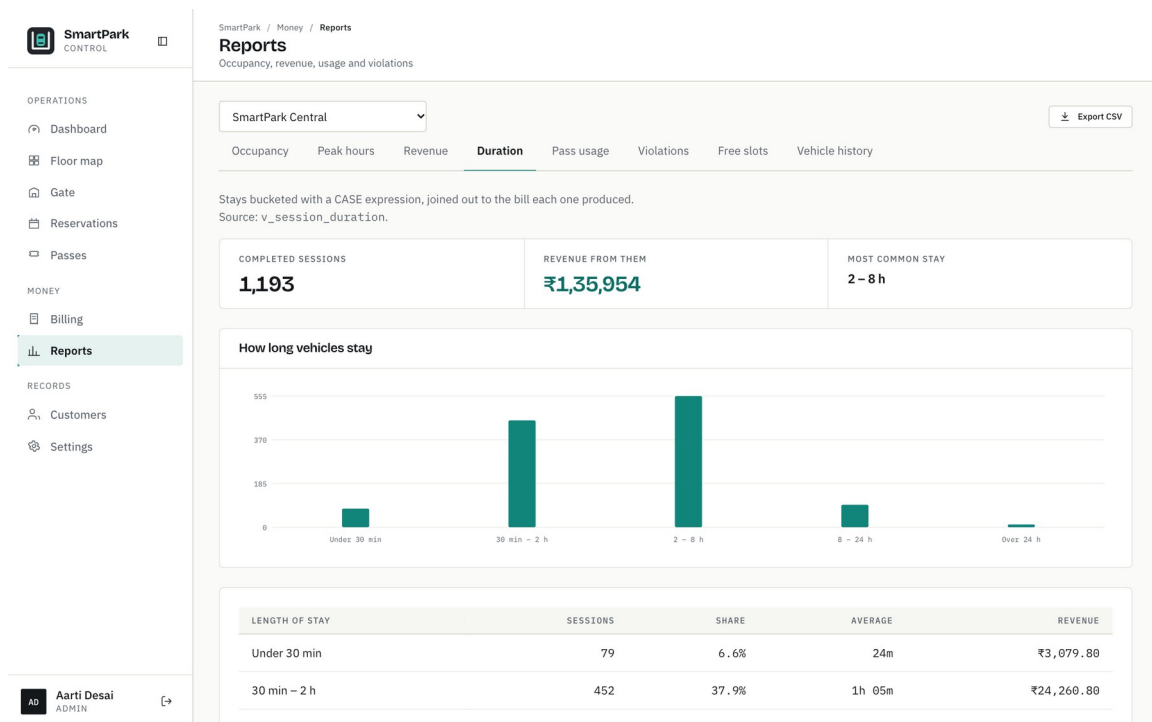


Figure 26: Report: duration (v_session_duration)

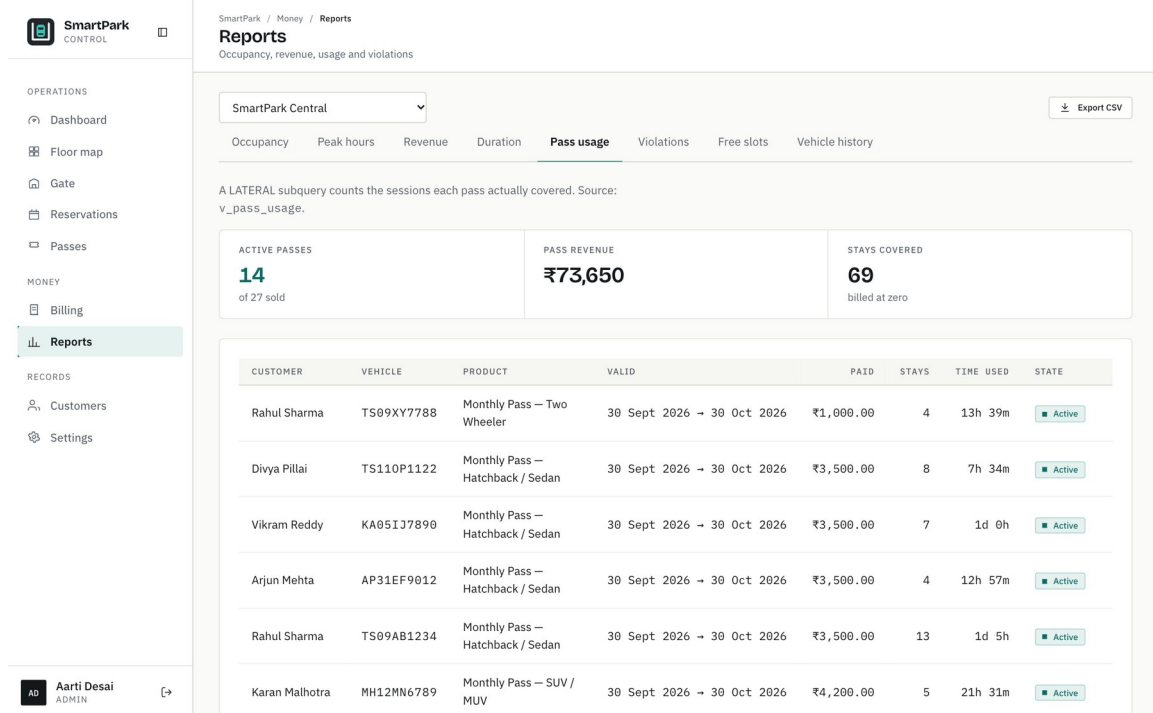


Figure 27: Report: pass usage (v_pass_usage)

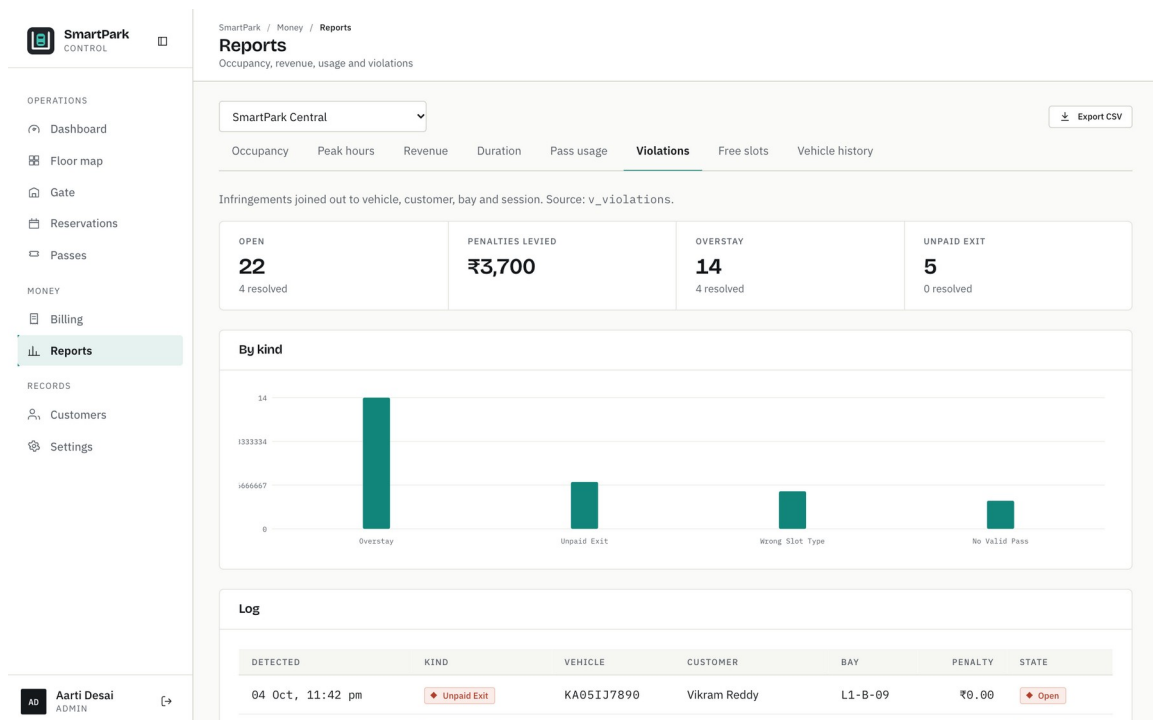


Figure 28: Report: violations (v_violations)

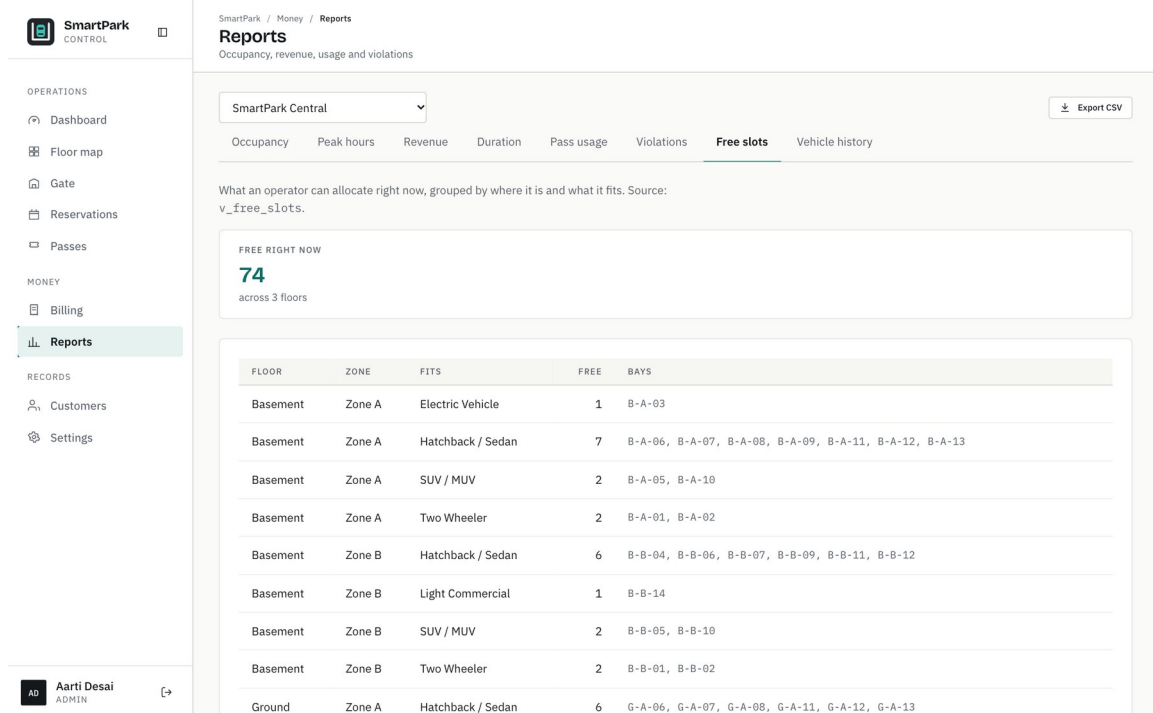


Figure 29: Report: free slots (`v_free_slots`)

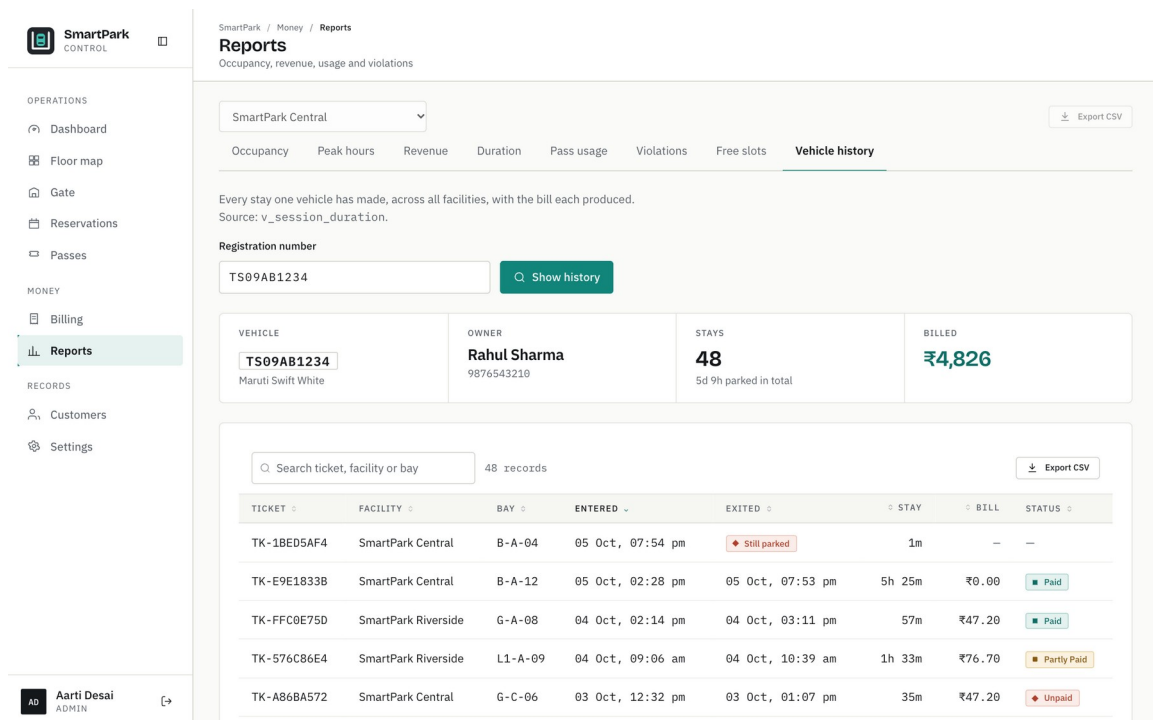


Figure 30: Report: complete history of one vehicle



SmartPark / Records / Customers

Customers

People and the vehicles they park

OPERATIONS

Dashboard

Floor map

Gate

Reservations

Passes

MONEY

Billing

Reports

RECORDS

Customers

Settings

AD Aarti Desai ADMIN

Customer register

Search, sort, export, or open a customer's vehicles.

+ Add customer

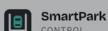
Search name, phone or email

31 records

Export CSV

NAME	PHONE	EMAIL	VEHICLES	CUSTOMER SINCE
Aditya Rane	9903456789	aditya.rane@example.com	2	05 Oct 2026
Ananya Ghosh	9856781234	ananya.ghosh@example.com	2	05 Oct 2026
Anil Varma	9885012347	anil.varma@example.com	0	05 Oct 2026
Arjun Mehta	9823456781	arjun.mehta@example.com	2	05 Oct 2026
Divya Pillai	9878123456	divya.pillai@example.com	2	05 Oct 2026
Farhan Ali	9803450123	farhan.ali@example.com	1	05 Oct 2026
Fatima Qureshi	9914567890	fatima.qureshi@example.com	2	05 Oct 2026
Gaurav Sinha	9825012345	gaurav.sinha@example.com	1	05 Oct 2026
Ishita Roy	9814501234	ishita.roy@example.com	1	05 Oct 2026
Joseph Mathew	9807234569	joseph.mathew@example.com	0	05 Oct 2026
Karan Malhotra	9867812345	karan.malhotra@example.com	2	05 Oct 2026

Figure 31: Customer register



SmartPark / Records / Customers

Customers

People and the vehicles they park

OPERATIONS

Dashboard

Floor map

Gate

Reservations

Passes

MONEY

Billing

Reports

RECORDS

Customers

Settings

AD Aarti Desai ADMIN

Customer register

Search, sort, export, or open a customer's vehicles.

+ Add customer

Search name, phone or email

31 records

Export CSV

NAME	PHONE	EMAIL	VEHICLES	CUSTOMER SINCE
Aditya Rane	9903456789	aditya.rane@example.com	2	05 Oct 2026
Ananya Ghosh	9856781234	ananya.ghosh@example.com	2	05 Oct 2026
Anil Varma	9885012347	anil.varma@example.com	0	05 Oct 2026
Arjun Mehta	9823456781	arjun.mehta@example.com	2	05 Oct 2026
Divya Pillai	9878123456	divya.pillai@example.com	2	05 Oct 2026
Farhan Ali	9803450123	farhan.ali@example.com	1	05 Oct 2026
Fatima Qureshi	9914567890	fatima.qureshi@example.com	2	05 Oct 2026
Gaurav Sinha	9825012345	gaurav.sinha@example.com	1	05 Oct 2026
Ishita Roy	9814501234	ishita.roy@example.com	1	05 Oct 2026
Joseph Mathew	9807234569	joseph.mathew@example.com	0	05 Oct 2026
Karan Malhotra	9867812345	karan.malhotra@example.com	2	05 Oct 2026

Vehicles — Aditya Rane

TS08YZ1616

TYPE

Hatchback / Sedan

VEHICLE

Kia Sonet Silver

STATE

Away

TS09UV7799

TYPE

Electric Vehicle

VEHICLE

MG ZS EV White

STATE

In lot

Register a vehicle

Registration number *

Vehicle type *

TS09AB1234

Two Wheeler

Make

Model

Colour

Maruti

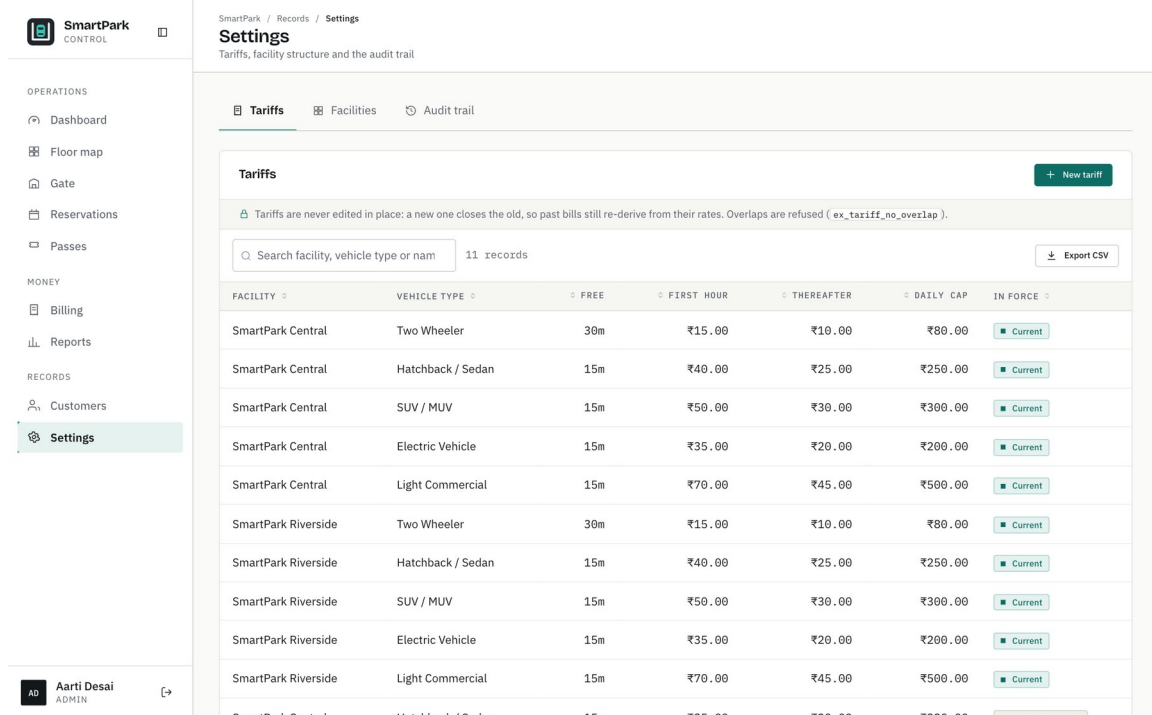
Swift

White

+ Register vehicle

Done

Figure 32: A customer's vehicles



SmartPark / Records / Settings
Settings
Tariffs, facility structure and the audit trail

Tariffs + New tariff

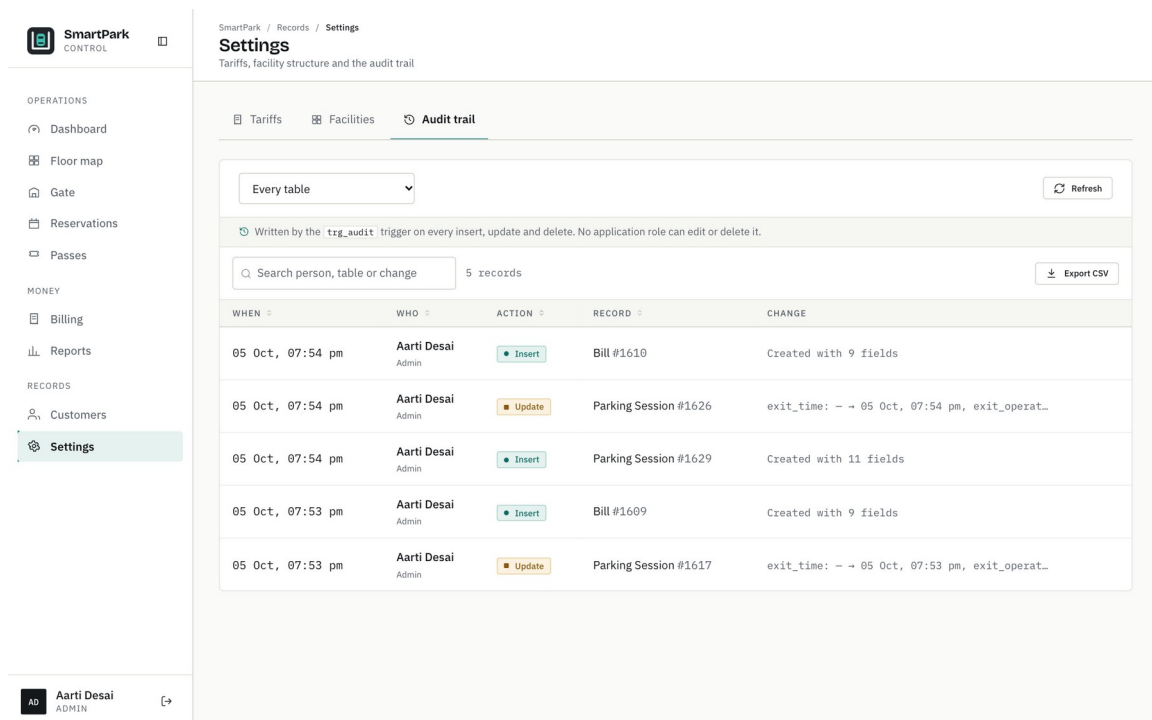
Tariffs are never edited in place: a new one closes the old, so past bills still re-derive from their rates. Overlaps are refused (ex_tariff_no_overlap).

Search facility, vehicle type or name 11 records Export CSV

FACILITY	VEHICLE TYPE	FREE	FIRST HOUR	THEREAFTER	DAILY CAP	IN FORCE
SmartPark Central	Two Wheeler	30m	₹15.00	₹10.00	₹80.00	Current
SmartPark Central	Hatchback / Sedan	15m	₹40.00	₹25.00	₹250.00	Current
SmartPark Central	SUV / MUV	15m	₹50.00	₹30.00	₹300.00	Current
SmartPark Central	Electric Vehicle	15m	₹35.00	₹20.00	₹200.00	Current
SmartPark Central	Light Commercial	15m	₹70.00	₹45.00	₹500.00	Current
SmartPark Riverside	Two Wheeler	30m	₹15.00	₹10.00	₹80.00	Current
SmartPark Riverside	Hatchback / Sedan	15m	₹40.00	₹25.00	₹250.00	Current
SmartPark Riverside	SUV / MUV	15m	₹50.00	₹30.00	₹300.00	Current
SmartPark Riverside	Electric Vehicle	15m	₹35.00	₹20.00	₹200.00	Current
SmartPark Riverside	Light Commercial	15m	₹70.00	₹45.00	₹500.00	Current

Aarti Desai ADMIN

Figure 33: Settings: versioned tariffs (administrators only)



SmartPark / Records / Settings
Settings
Tariffs, facility structure and the audit trail

Audit trail

Every table Refresh

Written by the trg_audit trigger on every insert, update and delete. No application role can edit or delete it.

Search person, table or change 5 records Export CSV

WHEN	WHO	ACTION	RECORD	CHANGE
05 Oct, 07:54 pm	Aarti Desai Admin	Insert	Bill #1610	Created with 9 fields
05 Oct, 07:54 pm	Aarti Desai Admin	Update	Parking Session #1626	exit_time: - - 05 Oct, 07:54 pm, exit_operat...
05 Oct, 07:54 pm	Aarti Desai Admin	Insert	Parking Session #1629	Created with 11 fields
05 Oct, 07:53 pm	Aarti Desai Admin	Insert	Bill #1609	Created with 9 fields
05 Oct, 07:53 pm	Aarti Desai Admin	Update	Parking Session #1617	exit_time: - - 05 Oct, 07:53 pm, exit_operat...

Aarti Desai ADMIN

Figure 34: Settings: audit trail written by the trg_audit trigger

9.3 Insert, View and Delete — Reflected in the Database

The sequence below inserts a customer through the interface, confirms the row in PostgreSQL, deletes it through the interface and confirms that the row is gone. The SQL query was run in psql

between the steps.

Before: the customer does not exist
 smartpark=# SELECT customer_id, full_name, phone, email, created_at::timestamp(0) FROM customer WHERE full_name = 'Ananya Reddy';
 customer_id | full_name | phone | email | created_at
 -----+-----+-----+-----+-----
 (0 rows)

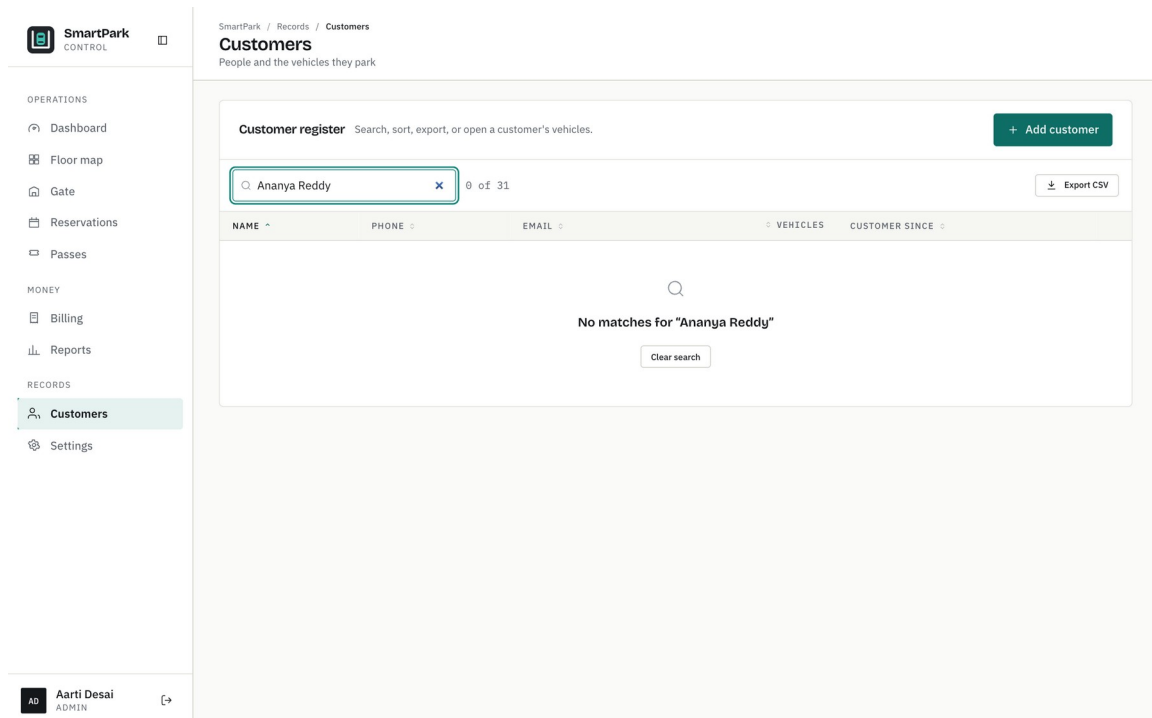


Figure 35: View before insertion: no customer named Ananya Reddy

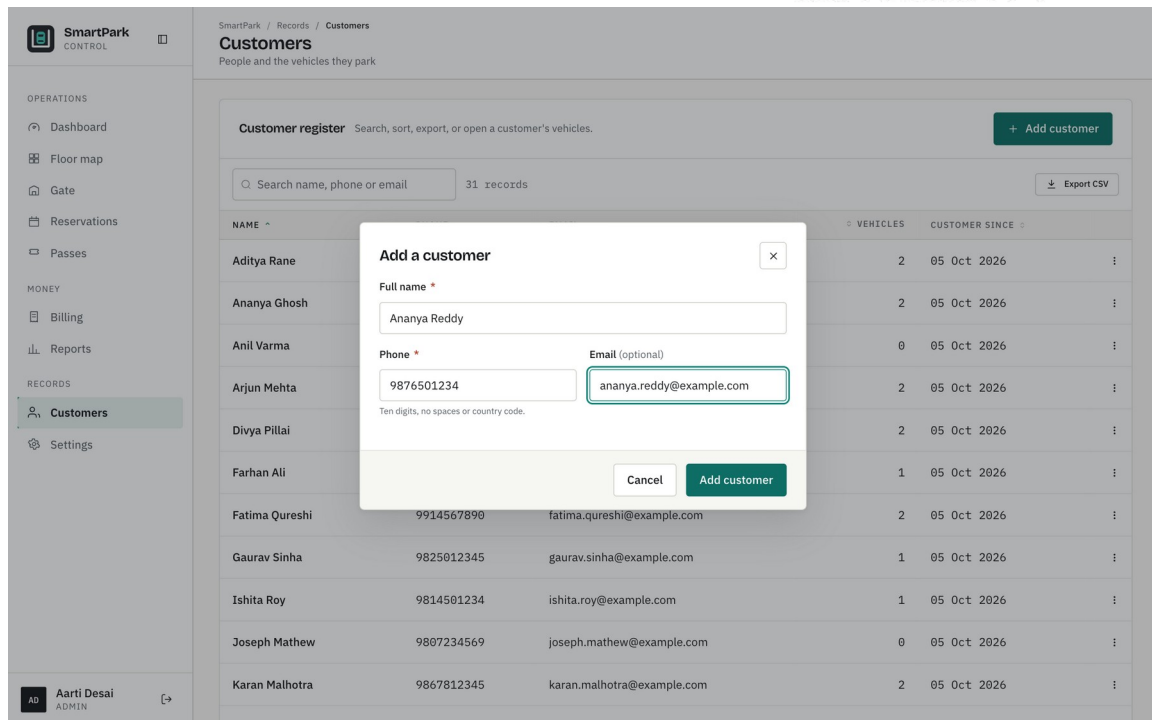


Figure 36: Insertion: the Add customer form

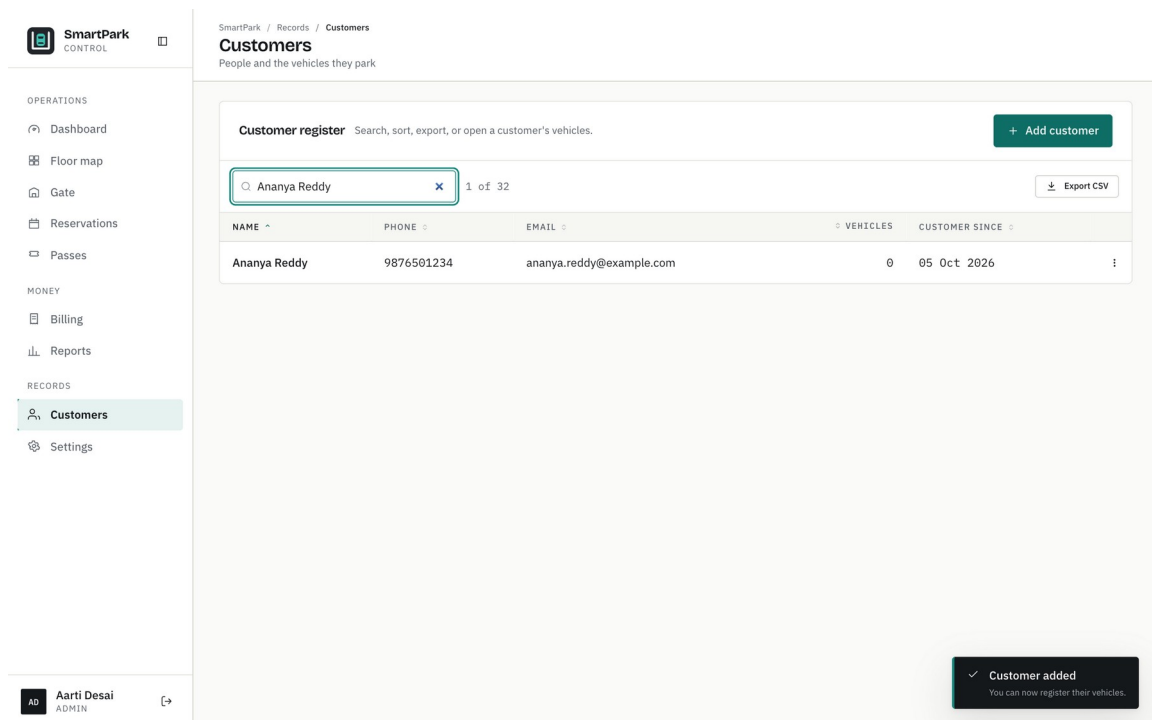


Figure 37: After insertion: the new customer is listed

After insertion: the row exists in the database

```
smartpark=# SELECT customer_id, full_name, phone, email, created_at::timestamp(0) FROM customer WHERE full_name = 'Ananya Reddy';
```

customer_id	full_name	phone	email	created_at
33	Ananya Reddy	9876501234	ananya.reddy@example.com	2026-10-05 20:27:01

(1 row)

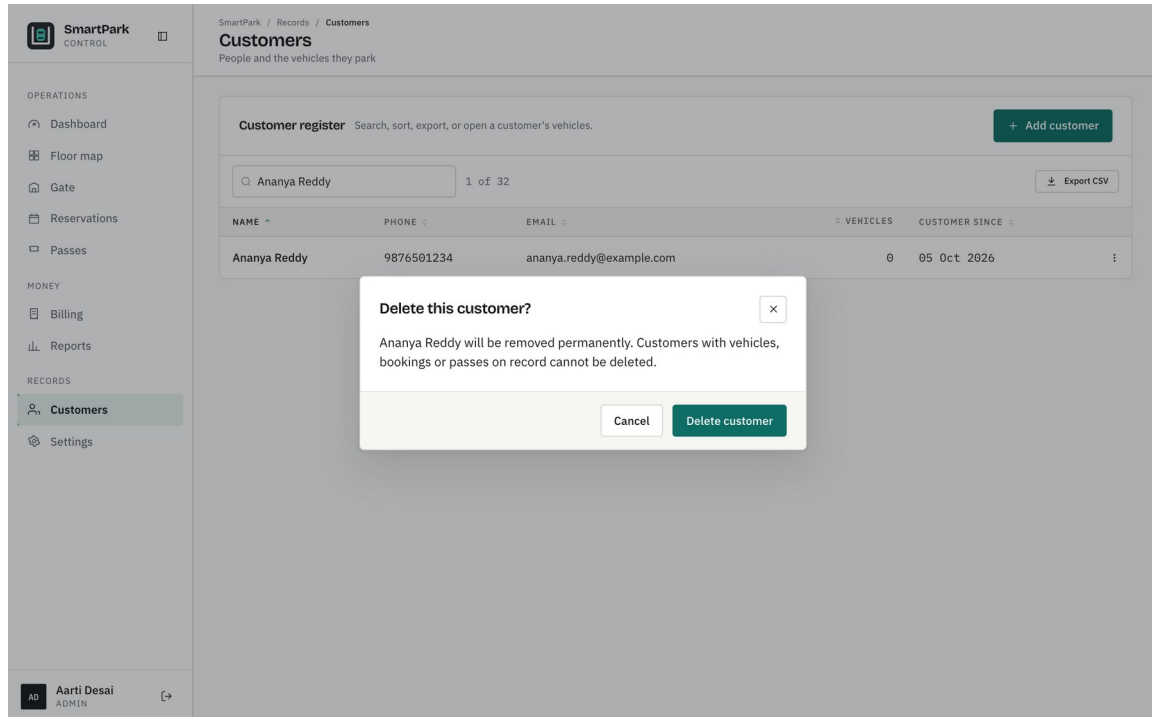


Figure 38: *Deletion: the confirmation dialog*

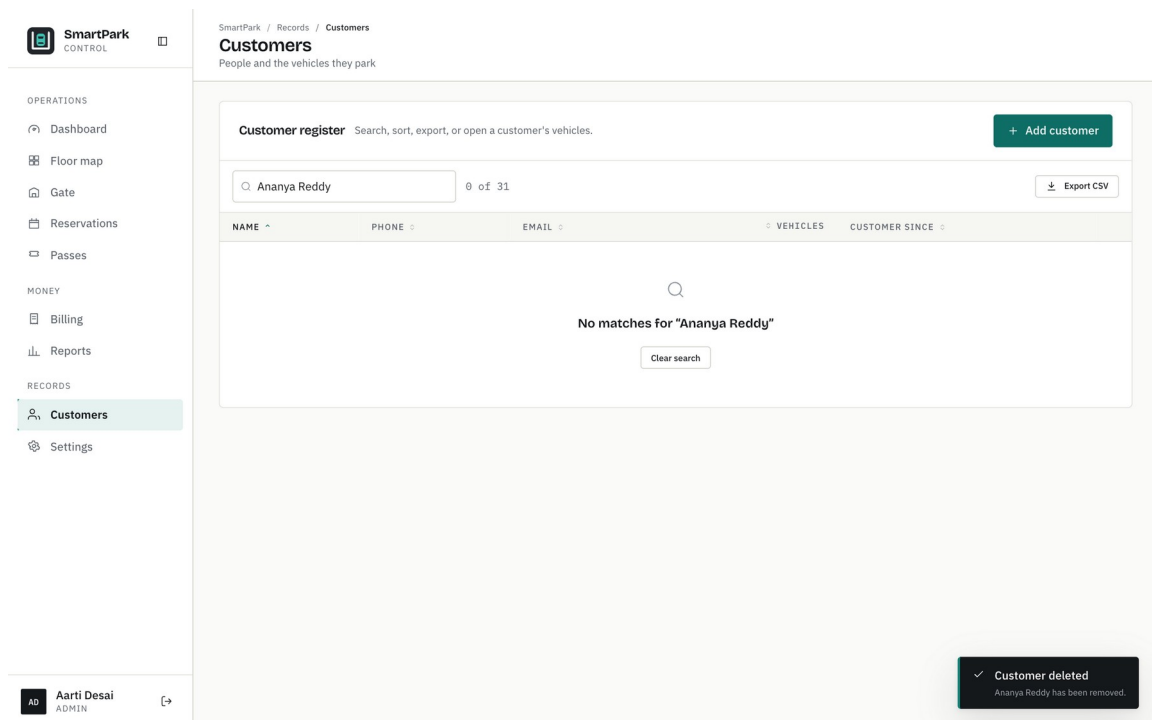


Figure 39: *After deletion: the customer is no longer listed*

After deletion: the row has been removed

```

smartpark=# SELECT customer_id, full_name, phone, email, created_at::timestamp(0) FROM customer WHERE full_name = 'Ananya Reddy';
 customer_id | full_name | phone | email | created_at
-----+-----+-----+-----+-----
(0 rows)
  
```

10. Implementation Details

10.1 Technology Stack and Architecture

The system has three layers. The **database layer** (PostgreSQL 17) holds the data and every business rule: tables and constraints, PL/pgSQL functions for the gate transactions and the charge calculation, triggers for derived values and auditing, views for the reports and row-level security policies. The **application layer** (Python 3.11, FastAPI) exposes 45 HTTP endpoints. They only pass requests to the database: no charge, allocation or status change is computed in Python. The **presentation layer** is HTML, CSS and JavaScript modules served by the same application.

Architecture

```

Browser (HTML + CSS + JavaScript modules)
  | fetch('/api/...') with a signed bearer token
  v
FastAPI application (api/main.py, api/db.py, api/auth.py, api/errors.py)
  | per request: BEGIN;
  |               SET LOCAL app.current_user_id = <user>;
  |               SET LOCAL ROLE parking_admin | parking_operator | parking_customer;
  |               <one statement or one function call>; COMMIT;
  v
PostgreSQL 17
- 17 tables, 36 foreign keys, 33 CHECK, 17 UNIQUE, 3 EXCLUDE constraints
- 16 PL/pgSQL functions, 14 triggers, 8 views, 59 indexes
- 26 row-level security policies on 9 tables

```

10.2 Database Connectivity

The application connects with **psycopg 3** through a connection pool. Every request runs inside one transaction that first declares who is asking, so the row-level security policies apply to real traffic and a pooled connection can never carry one user's identity into the next request.

api/db.py — connection pool and per-request transaction

```

_pool = ConnectionPool(DATABASE_URL, min_size=1, max_size=10, open=False,
                        kwargs={"row_factory": dict_row, "prepare_threshold": None})

ROLE_FOR = {"admin": "parking_admin", "operator": "parking_operator",
            "customer": "parking_customer"}

@contextmanager
def session_scope(user_id=None, role=None, *, privileged=False):
    """One transaction, with the caller's identity declared to the database."""
    with _pool.connection() as conn:
        with conn.transaction():
            with conn.cursor() as cur:
                if not privileged:
                    if user_id is not None:
                        cur.execute("SELECT set_config('app.current_user_id', %s, true)",
                                    (str(user_id),))
                    db_role = ROLE_FOR.get(role)
                    if db_role:
                        cur.execute(f"SET LOCAL ROLE {db_role}")
                yield cur

```

The database URL and the token-signing secret are read from environment variables

(SMARTPARK_DATABASE_URL, SMARTPARK_JWT_SECRET); no credentials are stored in the code.

10.3 Key Code Snippets

```
api/main.py — the gate-entry endpoint
@app.post("/api/gate/entry")
def gate_entry(body: GateEntryIn, user: dict = Depends(require_staff)):
    """One call, one transaction, one row lock. See fn_gate_entry in 006."""
    try:
        with database.session_scope(user["user_id"], user["role"]) as cur:
            cur.execute("SELECT * FROM fn_gate_entry(%s, %s, %s)",
                        (body.plate, body.facility_id, user["user_id"]))
            row = cur.fetchone()
            ...
            return ok(row)
    except Exception as exc:
        raise as_http(exc)          # PostgreSQL error -> plain-English message
```

```
006_gate_transactions.sql — slot allocation with a row lock
CREATE OR REPLACE FUNCTION fn_allocate_slot(p_facility_id BIGINT, p_vehicle_type_id BIGINT,
                                           p_at TIMESTAMPTZ DEFAULT now())
RETURNS BIGINT LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_temp AS $$
DECLARE v_slot_id BIGINT;
BEGIN
    SELECT s.slot_id INTO v_slot_id
    FROM slot s JOIN zone z ON z.zone_id = s.zone_id JOIN floor fl ON fl.floor_id = z.floor_id
    WHERE fl.facility_id = p_facility_id AND s.vehicle_type_id = p_vehicle_type_id AND s.is_active
    AND NOT EXISTS (SELECT 1 FROM parking_session ps
                    WHERE ps.slot_id = s.slot_id AND ps.exit_time IS NULL)
    AND NOT EXISTS (SELECT 1 FROM reservation r
                    WHERE r.slot_id = s.slot_id AND r.status IN ('held', 'confirmed')
                    AND tstzrange(r.reserved_from, r.reserved_until) @> p_at)
    ORDER BY fl.level_number, z.code, s.code
    LIMIT 1
    FOR UPDATE OF s SKIP LOCKED;      -- the lock: two operators never get the same bay
    RETURN v_slot_id;                -- NULL when the facility is full
END; $$;
```

```
005/017 — the trigger that makes every charge database-derived
CREATE OR REPLACE FUNCTION fn_bill_enforce_amounts() RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.base_amount := fn_calculate_charge(NEW.session_id);  -- replaces any client value
    NEW.tariff_id   := COALESCE(fn_applicable_tariff(NEW.session_id), NEW.tariff_id);
    ...
    NEW.tax_amount := ROUND(NEW.base_amount * v_tax_rate / 100.0, 2);
    IF NEW.base_amount + NEW.tax_amount = 0 AND NEW.status = 'unpaid' THEN
        NEW.status := 'paid';                                -- nothing owed
    END IF;
    RETURN NEW;
END; $$;
-- total_amount is a GENERATED ALWAYS column: base_amount + tax_amount
```

```
016 — no overpayment
CREATE TRIGGER trg_payment_within_balance BEFORE INSERT OR UPDATE OF amount, bill_id
ON payment FOR EACH ROW EXECUTE FUNCTION fn_payment_within_balance();
-- locks the bill row (FOR UPDATE) and refuses any payment beyond the amount still owed:
-- example refusal:
-- ERROR: That is more than the ₹236.00 still owed on this bill.
```

10.4 Security

- Passwords are stored only as **bcrypt** hashes; sign-in returns a signed, expiring token.

- Three PostgreSQL roles (parking_admin, parking_operator, parking_customer) and **26 row-level security policies** decide which rows each user can see or change; views are created with security_invoker so they cannot bypass the policies.
- All SQL uses bound parameters; no user input is ever concatenated into a statement.
- The audit_log table, written by a trigger on ten tables, records who changed what; no application role can modify it.

11. Testing

11.1 Approach

Testing focused on proving that the **database** refuses invalid data, not only that the interface behaves. Each test is a script in the repository that can be re-run, and the results below are captured from real runs (docs/TESTING.md).

Table 27: Test suites and results

Suite	What it proves	Result
Constraint tests (db/tests/constraint_tests.sql)	Each business rule rejects its violation	24 of 25 refused; 1 accepted and corrected by trigger, as designed
Lifecycle tests (db/tests/lifecycle_tests.sql)	Reservation expiry, overstay, pass cover and expiry, derived billing	11 of 11 pass
Row-level security tests (db/tests/rls_tests.sql)	Each role sees only permitted rows; fails closed	12 of 12 pass
End-to-end test (tests/e2e_smoke.py)	Sign-in → reserve → entry → exit → bill → payment → reports	40 of 40 checks pass
API tests (tests/test_api.py, pytest)	CRUD, ownership, servicing, payments, scoping	18 of 18 pass
Browser tests (Playwright)	Every page loads without errors; keyboard, dialogs, contrast	All pass

11.2 Constraint Test Cases

Table 28: Constraint test cases (each run in a rolled-back transaction)

#	Rule tested	Attempted violation	Database response
1	BUSINESS RULE 1: one active vehicle per slot	open a second session on a slot that is already occupied.	Refused: duplicate key value violates unique constraint "uq_active_session_slot"
2	BUSINESS RULE 1 (mirror): one active slot per vehicle	park an already-parked vehicle in a second free bay.	Refused: duplicate key value violates unique constraint "uq_active_session_vehicle"
3	BUSINESS RULE 2: slot / vehicle-type match	put a CAR into a BIKE bay.	Refused: insert or update on table "parking_session" violates foreign key constraint "fk_session_slot_type_match"
3b	BUSINESS RULE 2: lying about the vehicle type	claim a CAR is a BIKE so the slot type appears to match.	Refused: insert or update on table "parking_session" violates foreign key constraint "fk_session_vehicle_type_match"
4	BUSINESS RULE 3: exit after entry	stamp an exit one hour BEFORE the entry.	Refused: new row for relation "parking_session" violates check constraint "ck_session_exit_after_entry"
5	BUSINESS RULE 4: reservation window must be forward	reserve until BEFORE the reservation starts.	Refused: new row for relation "reservation" violates check constraint "ck_reservation_window"
6	BUSINESS RULE 4: two live holds may not overlap	book a slot for a window that overlaps a live hold.	Refused: conflicting key value violates exclusion constraint "ex_reservation_no_overlap"
7	BUSINESS RULE 5: a bill amount may not be negative	write a bill with a negative base amount.	Refused: new row for relation "bill" violates check constraint "ck_bill_base_non_negative"
8	BUSINESS RULE 5: a client cannot dictate the charge	insert a bill claiming the parking cost 1 rupee.	Accepted, amount corrected by trigger
9	Referential integrity: a session	reference a slot_id that does not	Refused: insert or update on table

#	Rule tested	Attempted violation	Database response
	needs a real slot	exist.	"parking_session" violates foreign key constraint "fk_session_slot_type_match"
10	Input validation: plate number format	register a vehicle with the plate "HELLO".	Refused: new row for relation "vehicle" violates check constraint "ck_vehicle_plate_shape"
11	Input validation: duplicate plate number	register a plate that already exists.	Refused: duplicate key value violates unique constraint "vehicle_plate_number_key"
12	Payment integrity: a payment must be positive	record a payment of zero.	Refused: new row for relation "payment" violates check constraint "ck_payment_amount_positive"
13	Tariff integrity: overlapping price lists	open a second tariff for a facility/type already priced.	Refused: conflicting key value violates exclusion constraint "ex_tariff_no_overlap"
14	Operator scoping: an operator must have a facility	create an operator with no facility_id.	Refused: new row for relation "app_user" violates check constraint "ck_app_user_operator_has_facility"
15	Pass integrity: two live passes on one vehicle	sell a second overlapping pass for the same vehicle/facility.	Refused: conflicting key value violates exclusion constraint "ex_pass_no_overlap"
16	Ownership: a booking must use the customer's own vehicle	reserve for customer A using customer B's car.	Refused: insert or update on table "reservation" violates foreign key constraint "fk_reservation_vehicle_owner"
17	Ownership: a pass must cover the customer's own vehicle	sell customer A a pass on customer B's car.	Refused: insert or update on table "parking_pass" violates foreign key constraint "fk_pass_vehicle_owner"
18	BUSINESS RULE 2 at booking time: bay / vehicle type	reserve a bay built for a different vehicle type.	Refused: insert or update on table "reservation" violates foreign key constraint "fk_reservation_slot_type_match"
19	Bay servicing: an out-of-service bay cannot be booked	take a free bay out of service, then reserve it.	Refused: That bay is out of service and cannot be reserved.
20	Bay servicing: an occupied bay cannot be taken out	as admin, take a bay with a parked car out of service.	Refused: Bay G-B-05 has a vehicle in it. Record its exit first.
21	Bay servicing: a note only describes an idle bay	attach a service note to a bay that is in service.	Refused: new row for relation "slot" violates check constraint "ck_slot_note_only_when_out"
22	Operator scoping: bays at another facility	operator posted to facility 1 services a facility 2 bay.	Refused: You can only manage bays at your own facility.
23	Audit trail: the history cannot be erased	as admin, delete rows from audit_log.	Refused: permission denied for table audit_log
24	Payment integrity: no paying past the balance	pay one rupee more than an unpaid bill's total.	Refused: That is more than the ₹224.20 still owed on this bill.

11.3 The Seven Required Integrity Checks

Table 29: Mapping of required integrity rules to tests

Requirement	Mechanism	Test
Two vehicles cannot occupy one slot	Partial unique index uq_active_session_slot	Test 1
One vehicle cannot occupy two slots	Partial unique index uq_active_session_vehicle	Test 2
Invalid vehicle / slot type combinations	Composite foreign keys on (slot_id, vehicle_type_id) and (vehicle_id, vehicle_type_id)	Tests 3, 3b, 18
Exit time before entry time	CHECK (exit_time > entry_time)	Test 4
Invalid or overlapping reservations	CHECK (reserved_until > reserved_from); GiST EXCLUDE	Tests 5, 6

Requirement	Mechanism	Test
Negative bill amounts	CHECK (base_amount >= 0) and positive payments	Tests 7, 12
Client-controlled billing values	Trigger fn_bill_enforce_amounts overwrites them	Test 8, Lifecycle 5

11.4 Lifecycle Test Results

Table 30: Time-driven rules, driven through the real gate functions

Scenario	Result	What the database did
1. A lapsed reservation expires and is logged as a no-show	PASS	reservation status is expired
	PASS	1 no-show violation logged
	PASS	running the sweep again changes nothing
2. A stay over 24 hours is billed and logged as an overstay	PASS	1800 minutes billed at Rs 489.70
	PASS	1 overstay violation, penalty Rs 200.00
	PASS	the bay is released (session closed)
3. A live pass makes the stay free and the bill settles itself	PASS	entry attached pass 36
	PASS	3-hour stay billed Rs 0.00, status paid
4. An expired pass no longer covers the stay	PASS	pass attached at entry: none
	PASS	same 3-hour stay now billed Rs 106.20, status unpaid
5. Billing is derived from the tariff, not typed in	PASS	base Rs 90.00 = fn_calculate_charge, tax Rs 16.20, total Rs 106.20

12. Conclusion and Future Enhancements

12.1 Conclusion

SmartPark meets the project's objectives. The relational schema of 17 tables is normalized to 3NF and BCNF, with its functional dependencies documented. The five business rules of the problem statement are enforced by the database through keys, CHECK and EXCLUDE constraints, triggers and transactions, and each was proven by a test that tries to break it. Slot allocation is safe under concurrent use because it locks the chosen row inside a transaction. Charges are always derived from the tariff, and the reports are SQL views over live data. A working web application connected to the database supports inserting, updating, deleting, searching and viewing records for three kinds of user, with row-level security limiting operators and customers to their own data.

The main lesson of the project is that rules placed in the database protect the data from every client, while rules placed only in application code protect it only from that one client.

12.2 Future Enhancements

- Automatic number-plate recognition at the gate, feeding `fn_gate_entry` directly.
- Integration with a payment gateway (UPI and cards) instead of recording payments manually.
- Scheduled expiry of reservations with `pg_cron` instead of the on-request sweep.
- Dynamic pricing by time of day and occupancy, as new tariff versions.
- A mobile application for customers to find and book bays.
- EV-charging bays with charging sessions billed separately.
- Occupancy prediction from the historical data in `parking_session`.

13. References

- [1] Silberschatz, A., Korth, H. F. and Sudarshan, S. (2019). *Database System Concepts*, 7th ed. McGraw-Hill.
- [2] Elmasri, R. and Navathe, S. B. (2016). *Fundamentals of Database Systems*, 7th ed. Pearson.
- [3] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
- [4] PostgreSQL Global Development Group (2024). *PostgreSQL 17 Documentation*.
<https://www.postgresql.org/docs/17/>
- [5] PostgreSQL Documentation: Row Security Policies.
<https://www.postgresql.org/docs/17/ddl-rowsecurity.html>
- [6] PostgreSQL Documentation: Exclusion Constraints and the btree_gist extension.
<https://www.postgresql.org/docs/17/btree-gist.html>
- [7] Psycopg 3 Documentation. <https://www.psycopg.org/psycopg3/docs/>
- [8] FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [9] Motion (animation library) Documentation. <https://motion.dev/>

14. Contribution of Each Member

All four members took part in the requirements discussion, reviewed the schema together and prepared for the presentation. The main responsibilities were divided as follows.

Table 31: *Contribution of each member*

Member	Roll number	Contribution
Sidharth Sood (project lead)	25WU0101132	Problem analysis and requirements; ER model and normalization to 3NF; PostgreSQL schema, constraints and indexes; PL/pgSQL gate-entry and gate-exit transactions, charge function and triggers; row-level security; FastAPI back end and database connectivity; deployment; integration of all modules and compilation of this report
Lakshay Sethia	25WU0101167	User-interface design and front-end screens (dashboard, floor map, gate, billing); connecting the screens to the API; UI screenshots for the report
Shardul Yadav	25WU0101127	SQL queries and the report views (occupancy, revenue, peak hours, duration, pass usage, violations, free slots); sample-data seed scripts; data dictionary
Naman Singla	25WU0101084	Testing: constraint, lifecycle and end-to-end test cases and their results; customer and vehicle CRUD testing; README, demo guide and presentation slides

Appendix A: GitHub Repository Link

Table 32: *Repository contents*

Item	Link / location
Source code repository	https://github.com/EXCALIBUR303/DBMS_SMART_PARKING
Live demonstration	https://smartpark-green-two.vercel.app
Database scripts	db/migrations/ (19 scripts, applied in order by setup.sh)
Test scripts and results	db/tests/, tests/, docs/TESTING.md
ER diagram, normalization, data dictionary	docs/database/
Screenshots	docs/screens/

A.1 Running the Project

```
git clone https://github.com/EXCALIBUR303/DBMS_SMART_PARKING.git
cd DBMS_SMART_PARKING
./setup.sh                                # creates the database, applies all migrations,
                                           # loads sample data, installs Python packages
./venv/bin/uvicorn api.main:app --port 8077
# open http://127.0.0.1:8077 and sign in as admin@smartpark.in / Parking@123
```